Inserção adaptativa de elementos de interface em malhas 2D

Operadores topológicos sobre estruturas de dados compactas baseadas em índices

Jeferson Coelho
Tecgraf/PUC-Rio*

Relatório técnico — versão 0.4 — outubro de 2026

Resumo

Este relatório documenta o primeiro protótipo, em 2D, de um algoritmo para inserir elementos de interface de espessura nula em malhas de elementos finitos representadas por estruturas de dados topológicas compactas (CHE e IBHM). O algoritmo recebe um conjunto de arestas e, para cada uma, cria um quadrilátero degenerado e repara a vizinhança de seus dois vértices aplicando um entre quatro operadores — **Canônico**, **Split Element**, **Expand Edge** e **Open Hole** —, escolhido exclusivamente a partir do *estado do vértice*. Descrevemos o problema, a estrutura IBHM, a arquitetura do algoritmo, cada operador com seus detalhes de implementação e o algoritmo de travessia, tal como implementados na classe `IBHMInterfaceElementBuilder`, que é a base a ser evoluída. Cada caso é ilustrado com figuras produzidas pelo próprio código. Em seguida, confrontamos a implementação com seus comentários e com o artigo da IBHM e listamos os defeitos corrigidos e os em aberto. Uma verificação experimental com 800 malhas aleatórias (triângulos, quadriláteros, malhas híbridas e polígonos de seis lados) confirma, em todas as configurações suportadas, a validade topológica, a contagem exata de vértices novos, as coordenadas das cópias e a independência da ordem de inserção (inclusive na inserção em lote); os cenários de verificação viraram testes automatizados (Google Test e CTest). A falha que resta é o vértice de borda com mais de uma aresta selecionada; as entradas inválidas, que também falhavam, passaram a ser rejeitadas.

*Texto redigido com o auxílio do Claude Code a partir do código do ramo `feature/edge-element-insertion` (commit `f4a5a83`), da apresentação `doc/ppt/Operators_Template` e do artigo da IBHM [1]. As figuras de malha foram geradas executando o próprio código (Apêndice A). As versões anteriores estão em `doc/report/historico/`.

Versão	Data	Mudanças
0.1	18/10/2026	Primeira versão, baseada na classe `CHEInterfaceElementBuilder`.
0.2	18/10/2026	Análise refeita sobre a classe `IBHMInterfaceElementBuilder`, base a ser evoluída; a implementação baseada na CHE deixa de ser considerada (a teoria da CHE foi mantida). Nova figura com os vetores da IBHM antes e depois de uma inserção; verificação com 800 malhas de triângulos, quadriláteros, híbridas e com polígonos, inclusive inserção em lote; novos casos-limite (índices inválidos, malha com furo interno); lista de defeitos restrita à IBHM.
0.3	18/10/2026	Sai o confronto com a apresentação de projeto, que não será mais mantida. Nova figura com as hipóteses x_1 e x_3 do Split Element. Comentários do código corrigidos (B1–B9, B11–B14) e reserva exata de memória em `reserveSpaceForNodes` com `addPoint` recebendo um `std::vector` (B2, parte do I4); defeitos atualizados (seção 7.4, antiga 7.5). Ferramentas de teste documentadas e explicadas no apêndice, com um teste novo para as coordenadas.
0.4	19/10/2026	Reflete as correções I3 (validação dos índices de entrada, O[ ] reservado com BORDER), I4 (cópias com as coordenadas do original, `duplicatePoint`), I5 (contagem exata, em duas etapas), I8 e I10 (includes; CMake sem Qt); nova tabela de defeitos corrigidos e I2 rebaixado a latente. Verificação com a V7 e nova seção de testes automatizados (Google Test e CTest), com controle negativo contra versões anteriores do núcleo. Apêndice reescrito para os testes, o código de apoio compartilhado com o `driver` e o novo build.

Sumário

1 Introdução **5**

1.1 Elementos de interface . . . 5
1.2 O problema da inserção . . . 5
1.3 Proposta e contribuições deste relatório . . . 6

2 Estruturas de dados: da CHE à IBHM **7**

2.1 Half-edges . . . 7
2.2 CHE: half-edges implícitas . . . 7
2.3 IBHM: malhas híbridas . . . 8
2.4 A classe `IBHM` do repositório . . . 8
2.5 Os vetores da IBHM durante uma inserção . . . 9

3 Arquitetura do algoritmo **10**

3.1 Princípios de projeto . . . 10
3.2 Componentes . . . 11
3.3 O gabarito do elemento de interface . . . 11
3.4 Estados de um vértice e escolha do operador . . . 12

4 Os operadores **14**

4.1 Canônico . . . 14
4.2 Split Element . . . 14
4.3 Expand Edge . . . 16
4.4 Open Hole . . . 16
4.5 Relações entre os operadores . . . 17
4.6 Contagem prévia de vértices . . . 18

5 O algoritmo de travessia **19**

5.1 Inserção de um lote de arestas . . . 19
5.2 O exemplo completo, passo a passo . . . 20
5.3 Propriedades observadas . . . 20

6 Verificação experimental **23**

6.1 Malhas aleatórias . . . 24
6.2 Casos específicos . . . 24
6.3 Testes automatizados . . . 24
6.4 O caso não suportado: vértice de borda com duas arestas . . . 25

7 Documentação × implementação **27**

7.1 Comentários do código . . . 27
7.2 Artigo da IBHM . . . 28
7.3 Arquivos de dados . . . 28
7.4 Defeitos e limitações encontrados no código . . . 28

8 Considerações finais e próximos passos **30**

8.1 Um invariante que resume o algoritmo . . . 30
8.2 Rumo ao 3D . . . 30
8.3 Próximos passos sugeridos . . . 31

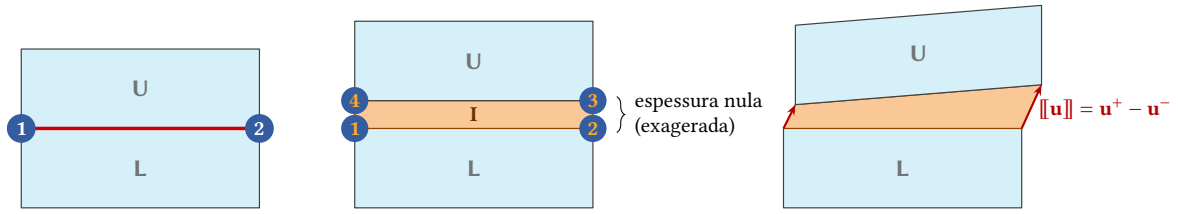
A Testes, ferramentas e geração das figuras **31**
A.1 Como compilar e rodar . . . 32
A.2 Os testes automatizados (`tests/`) . . . 32
A.3 O programa de teste (`driver.cpp`) . . . 33
A.4 Comparação de topologias (`compare.py`) . . . 34
A.5 Teste com malhas aleatórias (`stress.py`) . . . 34
A.6 Geração das figuras (`gen.py`) . . . 34

1 Introdução

1.1 Elementos de interface

Em muitos problemas de engenharia o comportamento que mais importa acontece em superfícies de descontinuidade: fraturas e falhas geológicas, juntas em maciços rochosos, contatos entre materiais distintos, caminhos de propagação de trincas. No método dos elementos finitos, uma forma clássica de representar essas descontinuidades é inserir, ao longo delas, *elementos de interface* — também chamados de elementos de espessura nula ou, quando associados a uma lei de tração–separação, elementos coesivos [6].

Em 2D, um elemento de interface linear é um quadrilátero de quatro nós cujos nós vêm aos pares geometricamente coincidentes (Figura 1). Dois nós pertencem a um lado da descontinuidade e dois ao outro. Como a espessura é nula, a grandeza cinemática relevante deixa de ser a deformação e passa a ser o *salto de deslocamento* entre os dois lados, $[\![\mathbf{u}]\!] = \mathbf{u}^+ - \mathbf{u}^-$, que mede abertura e deslizamento da interface.



(a) malha contínua (b) elemento de interface (c) abertura da interface

Figura 1: O elemento de interface. (a) Dois elementos contínuos compartilham uma aresta (em vermelho). (b) A aresta é *aberta*: os nós 1 e 2 são duplicados em 4 e 3, e entre U e L surge um quadrilátero de quatro nós com $\mathbf{x}_1 = \mathbf{x}_4$ e $\mathbf{x}_2 = \mathbf{x}_3$ (desenhado com espessura exagerada). (c) Durante a simulação os dois lados podem se separar; o elemento mede o salto de deslocamento.

Há duas estratégias para colocar esses elementos na malha. Na estratégia *a priori*, todos os elementos de interface são criados antes da análise, ao longo de todas as superfícies em que uma descontinuidade pode ocorrer. Na estratégia *adaptativa*, eles são inseridos durante a simulação, apenas onde e quando um critério (de tensão, de dano etc.) indica que a descontinuidade se formou [6]. A segunda estratégia evita a rigidez artificial e o custo de milhares de elementos inúteis, mas exige um algoritmo que altere a topologia da malha de forma incremental, local e eficiente, sobre uma malha que *já contém* elementos de interface.

1.2 O problema da inserção

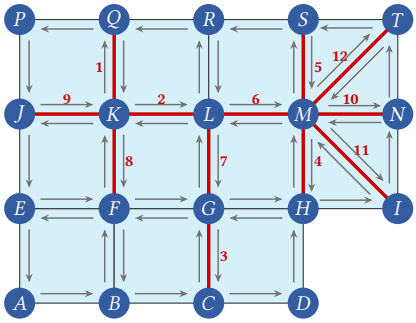
Problema

Dados: uma malha 2D orientada $\mathscr{M}$ e um conjunto S de arestas internas (cada uma compartilhada por dois elementos).
Obter: uma malha $\mathscr{M}'$ em que cada aresta de S foi *aberta*: os dois elementos que a compartilhavam passam a se ligar através de um novo elemento de interface de quatro nós, e os vértices são duplicados apenas o necessário para que a topologia continue consistente.

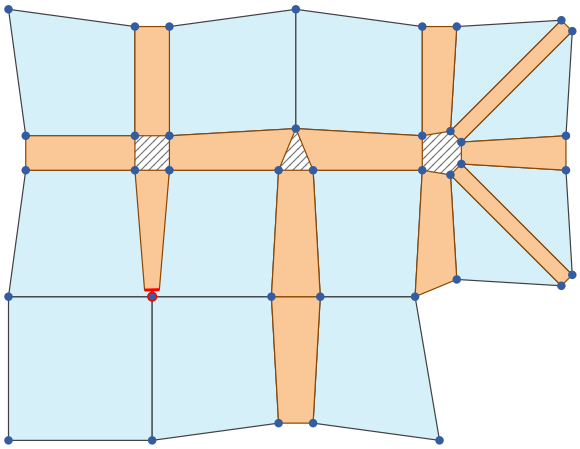
A Figura 2 mostra a entrada e a saída para o exemplo usado ao longo de todo o texto: uma malha híbrida de quadriláteros e triângulos, com 12 arestas selecionadas e numeradas na ordem de inserção. Parece simples, mas há várias dificuldades:

1. **Quantas cópias de cada vértice?** Um vértice interno tocado por uma única aresta selecionada não precisa ser duplicado geometricamente (a trinca termina nele). Com duas arestas, ele vira dois. Com três ou mais, surge uma *junção* (em T, em X, em estrela), e os elementos de interface não conseguem fechar a vizinhança sozinhos: aparece um *buraco* poligonal, visível como as regiões hachuradas na Figura 2b.

2. **Quais elementos vão para cada cópia?** A estrela do vértice precisa ser dividida em setores delimitados pelas arestas selecionadas, e cada setor deve passar a usar uma cópia diferente.
3. **Borda.** Na borda da malha a abertura não precisa ser fechada: o vértice sempre ganha uma cópia a mais.
4. **Incrementalidade.** Em uma análise adaptativa as arestas chegam aos poucos e em qualquer ordem. O resultado não deveria depender dessa ordem.
5. **Estruturas compactas.** Estruturas baseadas em índices, como a CHE e a IBHM [2, 1], armazenam a malha em vetores de inteiros, sem ponteiros. Os próprios autores da IBHM apontam como limitação o uso em remalhamento dinâmico [1]. Inserir elementos é exatamente esse tipo de alteração.
6. **Extensão para 3D.** O objetivo final é inserir elementos em faces de malhas 3D. As decisões tomadas em 2D devem ser transportáveis.



(a) Entrada: malha com as arestas selecionadas (em vermelho) e a ordem de inserção.



(b) Saída produzida pelo código: 12 elementos de interface (laranja), 19 vértices novos e três buracos (hachurados).

Figura 2: O exemplo da apresentação de projeto [7], executado pelo código. Na saída, cada cópia de um vértice foi deslocada um pouco na direção dos elementos que a usam, para que vértices coincidentes fiquem visíveis ("vista explodida"). A malha é a do arquivo `malha2Hyb.txt`.

1.3 Proposta e contribuições deste relatório

O algoritmo estudado aqui [7] parte de uma observação simples, que dá nome a um dos slides de projeto: embora o problema seja *abrir arestas e criar elementos*, as decisões difíceis acontecem nos *vértices*. Por isso, a inserção de cada aresta é decomposta em duas aplicações de operadores locais, uma em cada extremidade, e o operador é escolhido olhando apenas para o estado daquele vértice. Há quatro operadores (**Canônico**, **Split Element**, **Expand Edge** e **Open Hole**), e a ordem das arestas é livre.

Este relatório tem quatro objetivos:

- explicar o problema e a estrutura de dados IBHM de forma acessível, inclusive o que acontece com seus vetores a cada inserção (Seções 1 e 2);
- descrever a arquitetura do algoritmo, cada operador e o algoritmo de travessia exatamente como estão implementados na classe `IBHMInterfaceElementBuilder`, a base a ser evoluída (Seções 3 a 5);
- verificar experimentalmente o comportamento do código (Seção 6);
- confrontar os comentários do código e o artigo da IBHM com a implementação e listar os defeitos em aberto (Seção 7).

A Seção 8 reúne próximos passos, inclusive observações para a extensão 3D.

2 Estruturas de dados: da CHE à IBHM

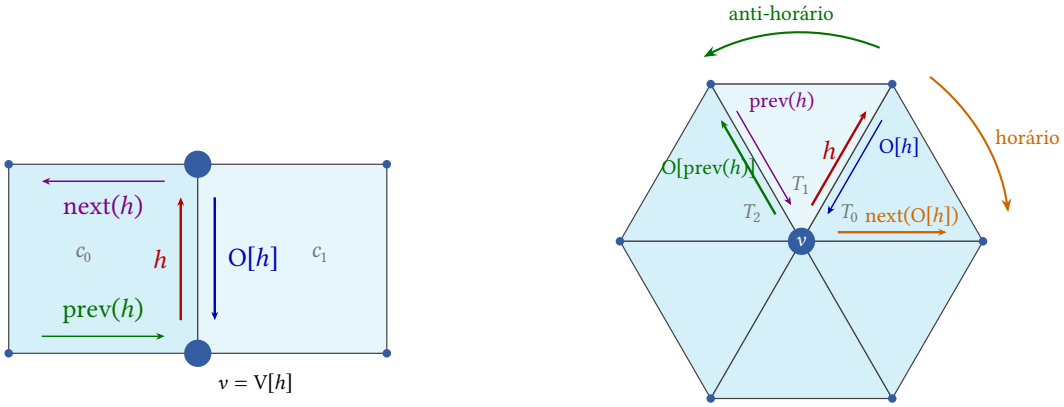
O algoritmo opera inteiramente sobre a topologia da malha. Esta seção apresenta o vocabulário necessário: o conceito de *half-edge*, a estrutura compacta CHE, sua extensão para malhas híbridas (IBHM) e como as duas estão implementadas no repositório.

2.1 Half-edges

Em uma malha 2D orientada, cada elemento é um polígono cujos vértices são listados no sentido anti-horário. Cada lado do elemento, visto com a orientação induzida pelo elemento, é uma *half-edge* [4]. Uma aresta interna é, portanto, representada por duas half-edges com sentidos opostos, uma em cada elemento vizinho; uma aresta de borda tem apenas uma. Cada half-edge h guarda o vértice de onde parte, V[h]. A partir dela definem-se três consultas (Figura 3a):

- next(h) e prev(h): a half-edge seguinte e a anterior no mesmo elemento;
- O[h]: a half-edge *oposta*, isto é, a mesma aresta vista pelo elemento vizinho (ou BORDER se h está na borda).

Note que h parte de V[h] e chega em V[next(h)], e que O[h] vai de V[next(h)] para V[h].



(a) Consultas básicas. **(b)** Girando em torno de um vértice v.

Figura 3: (a) Uma half-edge h do elemento c_0 e suas consultas. (b) As duas maneiras de girar em torno de um vértice usando apenas next, prev e O: a regra $h \leftarrow$ O[prev(h)] passa ao elemento vizinho no sentido anti-horário (usada por `reindexElementsCCW`), e a regra $h \leftarrow$ next(O[h]) passa ao vizinho no sentido horário (usada por `isBorder` e pelo segundo laço de `openHole`). Em ambos os casos a nova half-edge também parte de v.

2.2 CHE: half-edges implícitas

A *Compact Half-Edge* (CHE) [2] dispensa objetos "half-edge". Se todos os elementos têm k vértices, basta guardar a lista de elementos como um vetor de inteiros V[ ] com k entradas por elemento: a posição h desse vetor *é* a half-edge, e o valor armazenado é o vértice de origem. As demais consultas saem por aritmética:

$$\text{elem}(h) = \lfloor h/k \rfloor, \qquad \text{next}(h) = k\,\text{elem}(h) + (h+1) \bmod k, \qquad \text{prev}(h) = k\,\text{elem}(h) + (h+k-1) \bmod k.$$

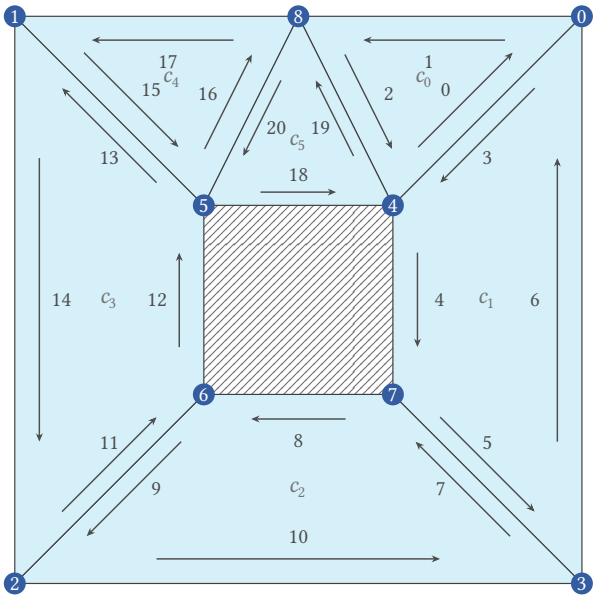
O único vetor adicional é o das opostas, O[ ], com uma entrada por half-edge. Toda a adjacência da malha cabe em dois vetores de inteiros do mesmo tamanho.

2.3 IBHM: malhas híbridas

A IBHM (*Index-Based data structure for Hybrid Meshes*) [1] estende a CHE (e a CHF [3], em 3D) para malhas com elementos de tipos diferentes. Em 2D, a mudança essencial é um vetor de *offsets* $Of[\,]$ que diz onde cada elemento começa em V[ ]. Assim, o elemento c ocupa as posições $Of[c], \ldots, Of[c] + n_v(c) - 1$, e:

- elem(h) é obtido por *busca binária* em $Of[\,]$, em $O(\log n_c)$ (em vez de uma divisão);
- next(h) $= Of[c] + \big((h - Of[c] + 1) \bmod n_v(c)\big)$ e analogamente para prev;
- O[ ] é idêntico ao da CHE;
- um vetor extra $VH[\,]$ associa a cada vértice uma half-edge que parte dele (a de borda, se o vértice estiver na borda), o que permite iniciar consultas de vizinhança e testar borda em $O(1)$.

A Figura 4 mostra os vetores para a malha do arquivo `meshIBHM.txt`, que reproduz (com índices a partir de zero) o exemplo da Figura 4 do artigo: um quadrado com um furo quadrado, formado por três quadriláteros e três triângulos.



he	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
$V[\,]$	4	0	8	0	4	7	3	3	7	6	2	2	6	5	1	1	5	8	5	4	8
$O[\,]$	3	–	19	0	–	7	–	5	–	11	–	9	–	15	–	13	20	–	–	2	16

$$Of[\,] = [0,\ 3,\ 7,\ 11,\ 15,\ 18,\ 21]$$

V[h]: vértice de origem de h; O[h]: oposta de h ("–" = BORDER); $Of[c]$: primeira half-edge do elemento c.

Figura 4: A malha de `meshIBHM.txt` com o índice de cada half-edge e os vetores da IBHM tal como construídos pela classe `IBHM`. O último elemento (c_5) começa num vértice diferente do usado no artigo, por isso suas três half-edges têm outra numeração; o restante coincide com a Figura 4 de [1] subtraindo-se 1 dos índices. O furo central é delimitado por half-edges de borda.

2.4 A classe `IBHM` do repositório

A classe `IBHM` (`core/TopologyDataStructures/IBHM.{h,cpp}`) segue as ideias acima, com algumas escolhas próprias que importam para o algoritmo de inserção (Tabela 1). As coordenadas são guardadas em `float`, e as malhas são lidas no formato de `malha2Hyb.txt`: número de vértices e de elementos, coordenadas e, para cada elemento, o número de vértices seguido dos índices.

Dois pontos merecem destaque. Primeiro, o estado COLLAPSED é uma extensão criada para o algoritmo de inserção: ele permite que um elemento de interface tenha um lado de comprimento zero sem vizinho,

Aspecto	Artigo da IBHM [1]	Classe `IBHM`
Índices	a partir de 1; O[h] = 0 na borda	a partir de 0; O[h] = BORDER = -1 (como `unsigned`: `0xFFFFFFFF`)
Estado extra de O	—	COLLAPSED = -2 marca uma aresta *colapsada* (de comprimento zero) de um elemento de interface
Offsets	n_c entradas	$n_c + 1$ entradas (sentinela final), de modo que $n_v(c) = Of[c+1] - Of[c]$
elem(h)	busca binária	busca binária com `std::upper_bound` nas $n_c + 1$ primeiras entradas de $Of[\,]$ (`offsetIterator`); next e prev também fazem essa busca
Construção de O	mapa indexado pelo par ordenado de vértices	para cada vértice, lista das half-edges que partem dele; procura-se a half-edge reversa (`buildOppositesLinear`)
Vetor $VH[\,]$	sim	**não**; o teste de borda (`isBorder`) gira em torno do vértice
Crescimento da malha	não previsto ("limitação" declarada)	`reserveSpaceForElements` aumenta os vetores de elementos (`resize`; as posições novas de O[ ] começam com BORDER) e `commitElement` valida cada novo elemento; `numberOfHalfEdges` devolve $Of[n_c]$, o número de half-edges válidas (sem o espaço reservado); `reserveSpaceForNodes` reserva capacidade exata para os pontos (`reserve`); `addPoint` acrescenta um ponto com as coordenadas dadas num `std::vector` (vazio = origem) e `duplicatePoint` acrescenta a cópia de um vértice existente, com as mesmas coordenadas

Tabela 1: Diferenças entre a IBHM do artigo e a classe `IBHM` do repositório.

e as consultas que giram em torno de vértices (`isBorder`) sabem pular esse lado. Segundo, a combinação "contar → reservar → validar" é a resposta da implementação à limitação apontada no artigo para alterações dinâmicas: o algoritmo de inserção calcula antes quantos vértices e elementos novos serão necessários (Seção 4.6), reserva o espaço de uma vez e só então altera a malha, sem realocações durante as operações.

A interface usada pelo construtor de elementos de interface é pequena: `heVertexIndex` (V), `heOpposite` (O), `heNext`, `hePrevious`, `heElement`, `isBorder`, `setElementVertex` (altera V[h]), `setOpposite` (altera O nos dois sentidos quando a oposta é uma half-edge válida, ou em um só sentido quando é BORDER ou COLLAPSED), `duplicatePoint`, `numberOfHalfEdges`, `nextAvailableHE` e `commitElement`. Nenhuma coordenada entra nas decisões: o algoritmo é puramente topológico, e as cópias criadas por `duplicatePoint` apenas herdam as coordenadas do vértice de origem.

2.5 Os vetores da IBHM durante uma inserção

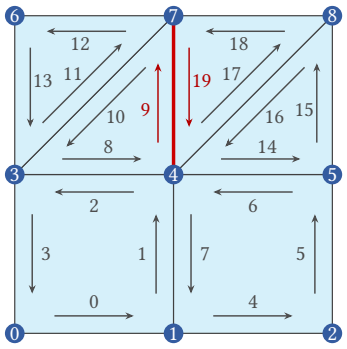
A Figura 5 mostra, numa malha híbrida pequena, o que acontece com os vetores quando uma aresta é aberta. A aresta 4–7 é compartilhada pelos triângulos que contêm as half-edges $a = 9$ ($4 \to 7$) e $b = 19$ ($7 \to 4$); o vértice 4 é interno e o vértice 7 está na borda.

1. **Reserva.** Antes de qualquer alteração, `reserveSpaceForElements(4, 1)` acrescenta 4 posições a V[ ] e a O[ ] (estas com BORDER) e uma entrada a $Of[\,]$, e `reserveSpaceForNodes(1)` reserva capacidade

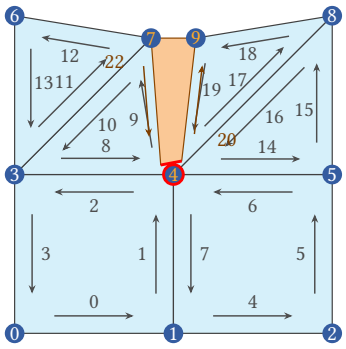
exata para um vértice, sem mudar o tamanho do vetor de coordenadas (a pré-contagem previu um vértice novo, para a borda).

2. **Validação antecipada.** `nextAvailableHE()` devolve $Of[n_c] = 20$, a primeira half-edge livre, e `commitElement(4)` já declara o elemento 6 válido, gravando $Of[7] = 24$. Na IBHM isso precisa acontecer *antes* dos operadores, porque next e prev das half-edges 20–23 dependem do offset do elemento.
3. **Operador em 4** (Canônico, interno): V[23] = V[20] = 4 e O[23] = COLLAPSED (marcado como C).
4. **Operador em 7** (Canônico, borda): cria o vértice 9, cópia de 7 (`duplicatePoint(7)`, mesmas coordenadas); V[21] = 9; a rotação a partir de b troca V[19] de 7 para 9 e para logo na borda; V[22] = 7 e O[21] = BORDER.
5. **Arestas do meio**: O[20] ↔ 19 e O[22] ↔ 9. As antigas opostas O[9] = 19 e O[19] = 9 são sobrescritas.

Ao todo, uma inserção altera poucas posições dos vetores antigos (em vermelho na figura) e preenche as quatro posições novas (em laranja). As posições reservadas de O[ ] começam com BORDER: até a v0.3 começavam com 0, um índice válido de half-edge, e um índice do espaço reservado passado como entrada era aceito como aresta (defeito I3, corrigido; Seção 7.4). No fluxo normal todas são sobrescritas pelos operadores e pela ligação das arestas do meio.



(a) Antes: a aresta 4–7.



(b) Depois: elemento 6 = [4, 9, 7, 4].

he	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
V antes	0	1	4	3	1	2	5	4	3	4	7	3	7	6	4	5	8	4	8	7				
V depois	0	1	4	3	1	2	5	4	3	4	7	3	7	6	4	5	8	4	8	**9**	**4**	**9**	**7**	**4**
O antes	–	7	8	–	–	–	14	1	2	19	11	10	–	–	6	–	17	16	–	9				
O depois	–	7	8	–	–	–	14	1	2	**22**	11	10	–	–	6	–	17	16	–	**20**	**19**	–	**9**	**C**

Of antes = [0, 4, 8, 11, 14, 17, 20] Of depois = [0, 4, 8, 11, 14, 17, 20, 24]

Figura 5: Uma inserção vista pelos vetores da IBHM. Na tabela, vermelho indica entradas antigas que mudaram e laranja as posições do novo elemento (half-edges 20–23); "–" é BORDER e C é COLLAPSED. O novo elemento é $[F, G, H, E] = [4, 9, 7, 4]$: colapsado em 4 (interno) e aberto em 7 (borda), onde surgiu o vértice 9.

3 Arquitetura do algoritmo

3.1 Princípios de projeto

A apresentação de projeto [7] lista como objetivos "um conjunto simples e pequeno de operadores", resolver a implementação "no nível dos operadores" e um algoritmo que percorra as arestas de entrada aplicando-os. A implementação materializa esses objetivos nas seguintes características:

P1. Baseado em operadores. Inserir uma aresta AB é sempre: criar o elemento, aplicar *um* operador em A, aplicar *um* operador em B e ligar as duas arestas "do meio" (Seção 3.3). Há apenas quatro operadores.

P2. Decisão centrada no vértice. O operador aplicado em um vértice depende só do estado desse vértice (pertence a algum elemento de interface? tem uma aresta colapsada? essa aresta compartilha um elemento com a nova aresta?). Não há análise de combinações de arestas nem dependência da ordem de chegada.

P3. Localidade. Cada operador altera somente a estrela de um vértice; o custo é $O(1)$ ou proporcional à valência.

P4. Puramente topológico. Nenhuma coordenada, ângulo ou produto vetorial é usado nas decisões: só V, O, next e prev. As cópias de um vértice apenas herdam as coordenadas do original. A apresentação justifica explicitamente essa escolha pela futura extensão a 3D.

P5. Duplicação adiada. O primeiro contato com um vértice interno *não* cria vértice novo: o elemento de interface ganha uma aresta *colapsada* (de comprimento zero) naquele vértice. A cópia só é criada quando, e se, uma segunda aresta chegar. É isso que faz uma ponta de trinca ficar fechada e que torna a contagem de vértices exata.

P6. Incremental. O construtor guarda o estado necessário entre chamadas; novas arestas podem ser inseridas a qualquer momento sobre uma malha que já tem elementos de interface (inserção adaptativa).

P7. Memória previsível. O número de vértices novos é calculado antes de alterar a malha, e todo o espaço é reservado de uma vez (Seção 4.6).

3.2 Componentes

A classe `IBHMInterfaceElementBuilder` (`core/InterfaceElementsBuilder`) recebe um ponteiro para a malha `IBHM` e mantém três estruturas auxiliares, que formam uma "segunda camada" de informação sobre a malha:

Estado	Significado
`_inInterfaceElement[v]`	o vértice v pertence a algum elemento de interface (por vértice).
`_isInterfaceElement[e]`	o elemento e é um elemento de interface (por elemento).
`_collapsed`	mapa $v \mapsto c$: vértices que têm uma aresta colapsada e a half-edge colapsada c (que vai de v para v).

Tabela 2: Estado mantido pelo construtor entre inserções.

Como esse estado não está guardado na malha, deve existir um único construtor por malha durante toda a sessão de edição; os comentários da classe registram essa restrição, ainda sem um mecanismo que a garanta (há um `@todo`). A classe declara como amiga a classe `BuilderInspector` (`tests/support`), que dá aos testes acesso de leitura a esse estado e ao operador escolhido; a biblioteca não a usa. Os métodos se dividem em quatro grupos:

- **travessia**: `insertInterfaceElements` valida e percorre as arestas de entrada (Seção 5);
- **decisão**: `retrieveOperator` escolhe o operador, com o auxílio de `retrieveAvailableVertex` para distinguir **Split Element** de **Expand Edge**;
- **operadores**: `canonical`, `splitElement`, `expandEdge` e `openHole`;
- **apoio**: `reindexElementsCCW` (troca o vértice de um leque de elementos) e `computeNumberOfNewVertices` (pré-contagem).

3.3 O gabarito do elemento de interface

Todo elemento de interface é criado com o mesmo gabarito (Figura 6). Seja a a half-edge selecionada, que vai de A para B no elemento U, e $b = \mathrm{O}[a]$, que vai de B para A no elemento L. O novo elemento recebe

as quatro próximas half-edges livres, $h_0, \ldots, h_3$, cujos vértices de origem são F, G, H, E:

- $h_0 : F \to G$ e $h_2 : H \to E$ são as arestas "do meio", paralelas a AB. Ao final, h_0 vira oposta de b e h_2 vira oposta de a; a e b deixam de ser opostas entre si.
- $h_3 : E \to F$ é o lado curto em A; E e F são cópias (topológicas) de A, E do lado de U e F do lado de L.
- $h_1 : G \to H$ é o lado curto em B; G é a cópia de B do lado de L, e H, do lado de U.

O operador do vértice A é chamado com o par (a, h_3) e o do vértice B com (b, h_1). Em ambos os casos vale a mesma convenção: *a origem de* `elementHE` *fica do lado do elemento que contém* `edgeHE`*, e* next(`elementHE`) *fica do outro lado.* Cada operador, portanto, só precisa decidir dois vértices, V[`elementHE`] e V[next(`elementHE`)], reindexar o que for preciso na estrela do vértice e definir O[`elementHE`]. As arestas do meio são sempre tratadas da mesma forma pela travessia, qualquer que seja o operador (slide "What happens in the middle?").

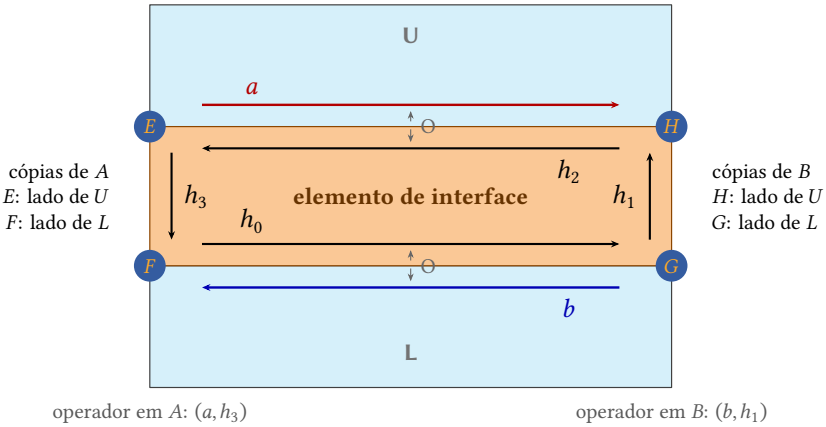


Figura 6: Gabarito de todo elemento de interface ($FGHE$, sentido anti-horário), desenhado com espessura exagerada. As arestas do meio h_0 e h_2 são ligadas a b e a pela travessia; os lados curtos h_3 (em A) e h_1 (em B) são responsabilidade dos operadores. Quando A e B são internos e ainda não pertencem a elemento de interface, o elemento resultante é $[A, B, B, A]$, com h_1 e h_3 colapsadas.

3.4 Estados de um vértice e escolha do operador

Do ponto de vista de um vértice original, a sequência de inserções o faz passar por uma pequena máquina de estados (Figura 7). Um vértice interno começa *regular*; a primeira aresta o deixa *colapsado* (um elemento de interface, nenhuma cópia nova); a segunda o *divide* em duas cópias, que ficam unidas pelos lados curtos dos dois elementos de interface; a terceira e as seguintes abrem e aumentam um *buraco*. Um vértice de borda é dividido já na primeira aresta, porque a abertura escapa para o exterior. O caso de uma segunda aresta em um vértice de borda não é tratado pela implementação atual (Seção 6.4).

A implementação não guarda esse estado explicitamente; ele é deduzido em `retrieveOperator` a partir das três estruturas da Tabela 2, olhando para o vértice v de onde parte a half-edge recebida (Figura 8):

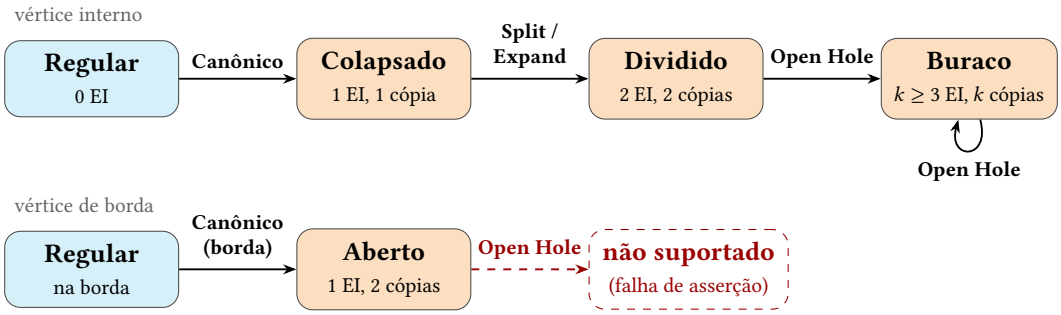


Figura 7: Estados de um vértice original ao longo das inserções (EI = elemento de interface). Os rótulos das transições são os operadores aplicados. Com k arestas selecionadas, um vértice interno termina com $\max(1, k)$ cópias e um de borda com $k + 1$ (este último só é atingido hoje para $k = 1$).

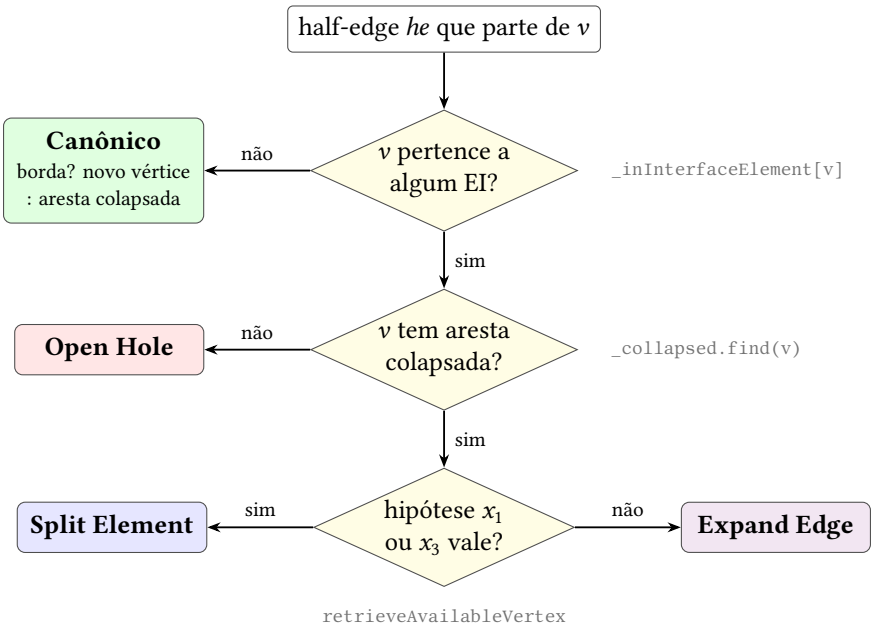


Figura 8: Árvore de decisão de `retrieveOperator`. As hipóteses x_1 e x_3 verificam se o elemento de interface que tem a aresta colapsada em v é vizinho de um dos dois elementos que contêm a aresta nova (Seção 4.2).

Operador	Condição em $v = \mathrm{V}[he]$	novos	V[eHE], V[next(eHE)]	O[eHE]	custo
Canônico (interno)	v fora de EI, interno	0	v, v	COLLAPSED	$O(d \log n_c)^*$
Canônico (borda)	v fora de EI, na borda	1 (v')	v', v	BORDER	$O(d \log n_c)$
Split Element (x_1)	colapsado, x_1 vale	1 (n)	n, v	c	$O(\log n_c)$
Split Element (x_3)	colapsado, x_3 vale	1 (n)	v, n	c	$O(\log n_c)$
Expand Edge	colapsado, nenhuma hipótese	1 (n)	v, n	c	$O(d \log n_c)$
Open Hole	em EI, sem colapsada	1 (n)	n, v	BORDER	$O(d \log n_c)$

Tabela 3: Resumo dos operadores. "eHE" é a half-edge `elementHE` do novo elemento (h_3 em A, h_1 em B); c é a half-edge colapsada registrada em `_collapsed[v]`, que deixa de ser colapsada e passa a ser a oposta de eHE. d é a valência do vértice e n_c o número de elementos: na IBHM cada next/prev faz uma busca binária em $Of[\,]$. *O custo do Canônico interno vem apenas do teste `isBorder`, que gira em torno do vértice.

4 Os operadores

Esta seção descreve os quatro operadores tal como implementados. Cada um recebe a half-edge da aresta nova que parte do vértice v (chamada `edgeHE` ou `he` no código) e a half-edge do lado curto correspondente do novo elemento (`elementHE`, h_3 ou h_1). Nos pseudocódigos, $\mathrm{V}[h] \leftarrow w$ corresponde a `setElementVertex(h, w)`, "$\mathrm{O}[x] \leftrightarrow y$" a `setOpposite(x, y)` com y uma half-edge válida (liga nos dois sentidos) e `duplicatePoint`(v) cria um vértice novo, cópia de v, com as mesmas coordenadas.

As figuras desta seção foram geradas executando o código em malhas pequenas (Apêndice A). Convenções visuais: elementos contínuos em azul, elementos de interface em laranja, aresta a inserir em vermelho; vértices com aresta colapsada têm contorno vermelho; vértices que pertencem a algum elemento de interface têm rótulo laranja; as cópias de um vértice X são chamadas $X_1, X_2, \ldots$ na ordem de criação e são desenhadas ligeiramente afastadas umas das outras. Arestas colapsadas (comprimento zero) aparecem como um pequeno segmento vermelho, com uma espessura *apenas visual*.

Todos os operadores usam a rotina de apoio `reindexElementsCCW` (Algoritmo 1), que gira no sentido anti-horário a partir de uma half-edge trocando o vértice de origem até encontrar uma half-edge de parada, a borda ou o ponto de partida.

Algoritmo 1 `reindexElementsCCW`(h, stop, w)

1: $s \leftarrow h$
2: **repita**
3: $\quad \mathrm{V}[h] \leftarrow w$ ▷ o canto de h passa a usar o vértice w
4: $\quad h \leftarrow \mathrm{O}[\mathrm{prev}(h)]$ ▷ próximo elemento no sentido anti-horário
5: **até que** $h = \mathit{stop}$ **ou** $h = \textsc{border}$ **ou** $h = s$

4.1 Canônico

Quando: o vértice v ainda não pertence a nenhum elemento de interface. É a "inicialização" do vértice.

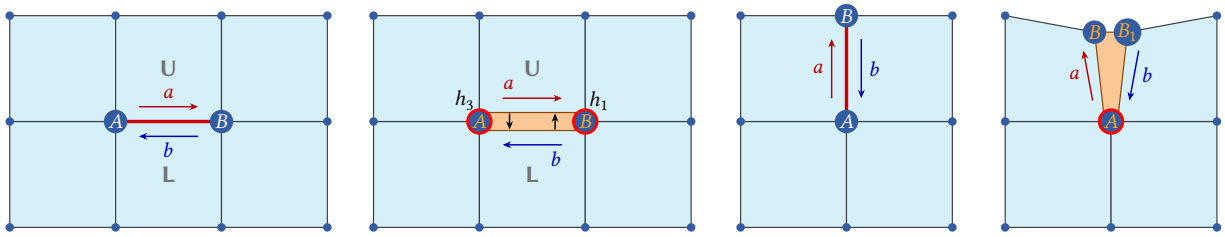
O que faz: se v é interno, nada é duplicado: o lado curto do novo elemento vira uma aresta colapsada de v para v, com oposta COLLAPSED, e ela é registrada em `_collapsed[v]`. A trinca termina em v e o elemento de interface fica "fechado" ali. Se v está na borda, a abertura chega ao exterior: cria-se um vértice v', todos os elementos do lado de `edgeHE` até a borda passam a usá-lo (rotação anti-horária), e o lado curto do novo elemento fica na borda (Algoritmo 2 e Figura 9).

Algoritmo 2 `canonical`(e, x) (e = `edgeHE`, x = `elementHE`)

1: $v \leftarrow \mathrm{V}[e]$
2: **se** `isBorder`(e) **então** ▷ gira em torno de v procurando uma half-edge sem oposta
3: $\quad v' \leftarrow$ `duplicatePoint`(v)
4: $\quad \mathrm{V}[x] \leftarrow v'$
5: $\quad$ `reindexElementsCCW`$(e, \textsc{border}, v')$ ▷ lado de e, até a borda
6: $\quad \mathrm{O}[x] \leftarrow \textsc{border}$; $\quad$ `_inInterfaceElement`$[v'] \leftarrow$ **true**
7: **senão**
8: $\quad \mathrm{V}[x] \leftarrow v$; $\quad \mathrm{O}[x] \leftarrow \textsc{collapsed}$; $\quad$ `_collapsed`$[v] \leftarrow x$
9: $\mathrm{V}[\mathrm{next}(x)] \leftarrow v$; $\quad$ `_inInterfaceElement`$[v] \leftarrow$ **true**

4.2 Split Element

Quando: v tem uma aresta colapsada c (pertence a exatamente um elemento de interface I, criado pelo Canônico) *e* I é vizinho de um dos dois elementos que contêm a aresta nova. Esse elemento comum é o *elemento compartilhado* S.



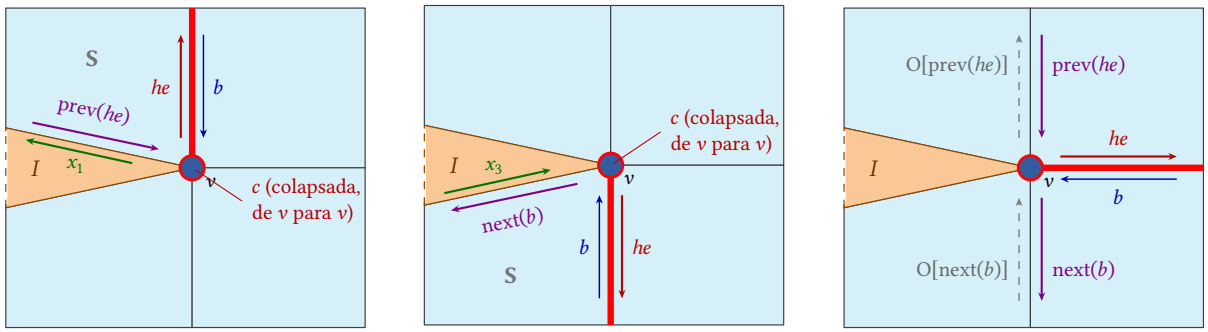
(a) Antes: A e B internos. **(b)** Depois: elemento $[A, B, B, A]$. **(c)** Antes: B na borda. **(d)** Depois.

Figura 9: Operador Canônico. (a–b) Com os dois vértices internos, o elemento de interface tem as duas extremidades colapsadas (h_3 em A e h_1 em B, em vermelho) e nenhum vértice é criado: geometricamente ele é só a aresta AB. (c–d) Com B na borda, B é duplicado em B_1; o operador em B recebe b, e os elementos do lado de b (à direita) passam a usar a cópia; o lado curto em B é uma aresta de borda. Em A (interno) a extremidade continua colapsada.

Como o código testa (`retrieveAvailableVertex`, Algoritmo 3). Seja he a half-edge nova que parte de v e $b = \mathrm{O}[he]$. Há duas posições possíveis para S:

- **hipótese** x_1: S é o elemento de he. Então a aresta de S que chega em v, $\mathrm{prev}(he)$, é vizinha de I, e sua oposta $x_1 = \mathrm{O}[\mathrm{prev}(he)]$ é o lado longo de I que *sai* de v logo depois de c, isto é, $\mathrm{prev}(x_1) = c$;
- **hipótese** x_3: S é o elemento de b. Então a aresta de S que sai de v, $\mathrm{next}(b)$, é vizinha de I, e $x_3 = \mathrm{O}[\mathrm{next}(b)]$ é o lado longo de I que *chega* em v logo antes de c, isto é, $\mathrm{next}(x_3) = c$.

Qual hipótese ocorre não depende de qual half-edge da aresta o usuário escolheu, e sim da posição da aresta nova em relação a I: em S, se a aresta nova vem depois da aresta de I (no sentido anti-horário de S) vale x_1; se vem antes, vale x_3. A Figura 10 mostra, em detalhe, as consultas feitas em cada caso e também o caso em que nenhuma das duas vale; a Figura 11 mostra o resultado numa malha.



(a) Hipótese x_1: S é o elemento de he. $x_1 = \mathrm{O}[\mathrm{prev}(he)]$ e $\mathrm{prev}(x_1) = c$.

(b) Hipótese x_3: S é o elemento de b. $x_3 = \mathrm{O}[\mathrm{next}(b)]$ e $\mathrm{next}(x_3) = c$.

(c) Nenhuma: os opostos caem fora de I. ⇒ Expand Edge.

Figura 10: As consultas de `retrieveAvailableVertex` em torno de um vértice v que tem uma aresta colapsada c (o elemento de interface I, à esquerda, termina em v; a aresta nova está em vermelho grosso). (a) A aresta nova sobe: o elemento de he tem uma aresta, $\mathrm{prev}(he)$, colada ao lado longo de I que sai de v logo depois de c. (b) A aresta nova desce: agora é o elemento de b que encosta em I, pela aresta $\mathrm{next}(b)$, colada ao lado longo de I que chega em v logo antes de c. (c) A aresta nova continua a trinca: os dois testes levam a elementos contínuos, e o operador escolhido é o Expand Edge. Em (a) e (b), o vértice novo vai para S.

O que faz (Algoritmo 3): cria um vértice n e o coloca "dentro" de S: só dois cantos mudam, o canto de S em v e o canto de I adjacente a S. A antiga aresta colapsada c deixa de ter comprimento zero (passa a ligar v e n) e vira a fronteira entre I e o novo elemento, sendo ligada a `elementHE`. O custo é um número constante de passos ($O(\log n_c)$ na IBHM, por causa das consultas next/prev).

Algoritmo 3 `retrieveAvailableVertex`(he) e `splitElement`(he, av, sh, x)

função RETRIEVEAVAILABLEVERTEX(he)
  $v \leftarrow \mathrm{V}[he]$; $b \leftarrow \mathrm{O}[he]$; $c \leftarrow$ `_collapsed`$[v]$
  $x_1 \leftarrow \mathrm{O}[\mathrm{prev}(he)]$
  se $x_1 \neq$ BORDER **e** $\mathrm{prev}(x_1) = c$ **então retorne** $(av = x_1,\ sh = he)$ ▷ hipótese x_1
  $x_3 \leftarrow \mathrm{O}[\mathrm{next}(b)]$
  se $x_3 \neq$ BORDER **e** $\mathrm{next}(x_3) = c$ **então retorne** $(av = c,\ sh = \mathrm{next}(b))$ ▷ hipótese x_3
  retorne (BORDER, BORDER) ▷ nenhum elemento compartilhado ⇒ Expand Edge

procedimento SPLITELEMENT(he, av, sh, x)
  $v \leftarrow \mathrm{V}[he]$; $n \leftarrow$ `duplicatePoint`(v)
  $\mathrm{V}[av] \leftarrow n$ ▷ canto do elemento de interface antigo
  $\mathrm{V}[sh] \leftarrow n$ ▷ canto do elemento compartilhado
  se $he = sh$ **então** ▷ hipótese x_1
    $\mathrm{V}[x] \leftarrow n$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow v$; $\mathrm{O}[x] \leftrightarrow \mathrm{prev}(av)$ ▷ $\mathrm{prev}(av) = c$
  senão ▷ hipótese x_3
    $\mathrm{V}[x] \leftarrow v$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow n$; $\mathrm{O}[x] \leftrightarrow av$ ▷ $av = c$
  marca v e n como vértices de EI; remove v de `_collapsed`

4.3 Expand Edge

Quando: v tem uma aresta colapsada c, mas nenhuma das duas hipóteses vale: o elemento de interface I não toca os elementos da aresta nova. É o caso típico de uma trinca que continua "em frente" através de v (Figura 12).

O que faz (Algoritmo 4): a estrela de v precisa ser dividida em dois setores, delimitados pela aresta nova de um lado e por I do outro. O operador cria n e gira no sentido anti-horário *a partir da aresta colapsada* c: primeiro troca o canto de c em I, depois sai de I pelo seu lado longo $\mathrm{prev}(c)$ e percorre os elementos daquele setor até alcançar he, que *não* é alterado. Assim, a parada natural da rotação é justamente cruzar a aresta nova. A aresta c deixa de ser colapsada e é ligada a `elementHE`.

Algoritmo 4 `expandEdge`(he, x)

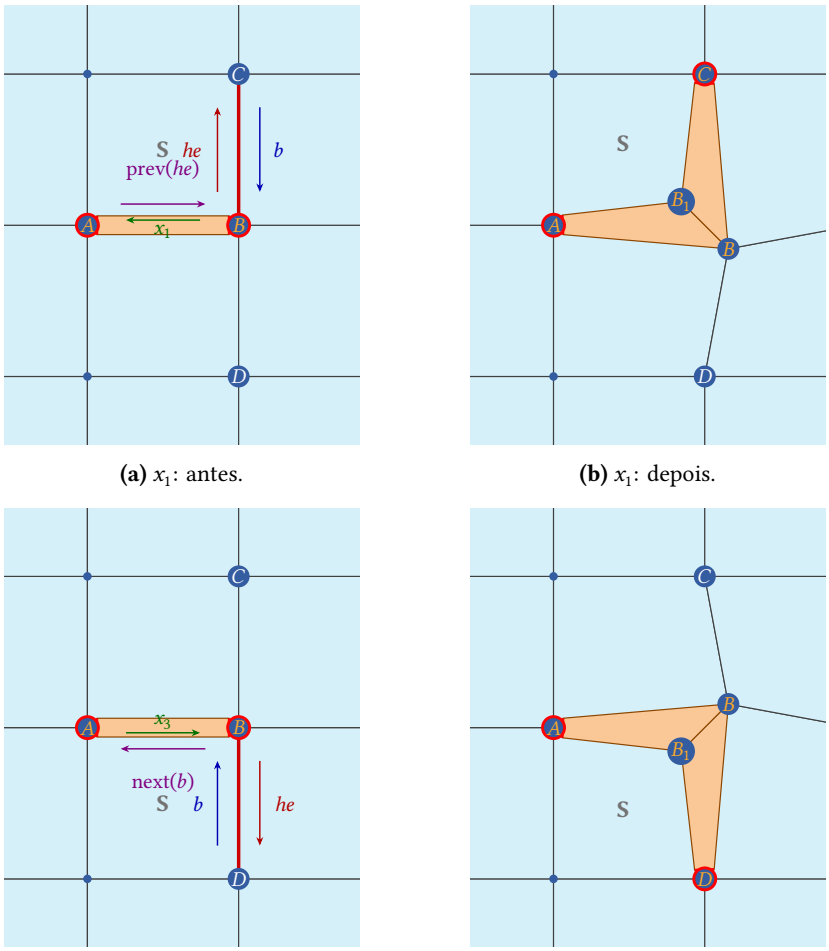
$v \leftarrow \mathrm{V}[he]$; $n \leftarrow$ `duplicatePoint`(v); $c \leftarrow$ `_collapsed`$[v]$
`reindexElementsCCW`(c, he, n) ▷ de c até he (exclusive)
$\mathrm{O}[c] \leftrightarrow x$
$\mathrm{V}[x] \leftarrow v$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow n$
marca v e n como vértices de EI; remove v de `_collapsed`

4.4 Open Hole

Quando: v já pertence a um elemento de interface, mas não tem aresta colapsada. Isso acontece a partir da terceira aresta em um vértice interno: depois do Split/Expand o vértice original já virou duas cópias, v e w, e os dois elementos de interface J e K que passam por ele estão colados pelos seus lados curtos (um vai de v para w, o outro de w para v).

O que faz (Algoritmo 5): com três ou mais elementos de interface chegando ao mesmo ponto, os lados curtos não conseguem mais fechar a vizinhança, e aparece um buraco. O operador cria n e faz duas travessias a partir de he:

1. no sentido *anti-horário*, troca v por n em todos os elementos até encontrar o primeiro elemento de interface J; troca também o canto de J e "descola" o lado curto de J que chega a esse canto



(a) x_1: antes. **(b)** x_1: depois.

(c) x_3: antes. **(d)** x_3: depois.

Figura 11: Split Element em B. Nos dois casos já existe um elemento de interface em AB, colapsado em A e em B. (a–b) A aresta nova BC sobe: o elemento compartilhado S (acima de AB) contém he, e o lado de I que sai de B é x_1. (c–d) A aresta nova BD desce: agora S (abaixo de AB) contém b, e o lado de I que chega em B é x_3. Em ambos, só S e um canto de I passam a usar a nova cópia B_1, e a antiga aresta colapsada em B vira o lado comum entre os dois elementos de interface. Note que C e D, internos, ficam colapsados.

(O ← BORDER);

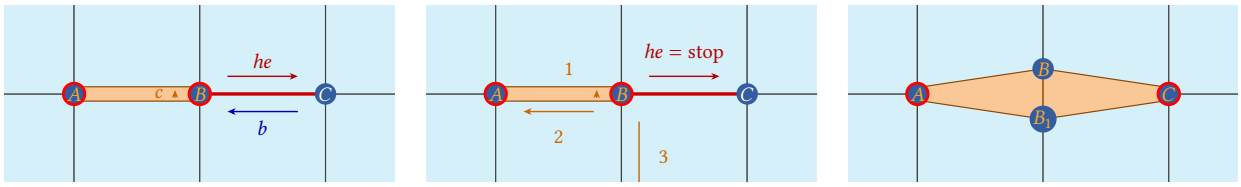
2. no sentido *horário* (regra $h \leftarrow \text{next}(\text{O}[h])$), apenas procura o primeiro elemento de interface K e descola o lado curto de K que sai de v.

O lado curto do novo elemento também fica na borda. Na primeira vez, os lados descolados de J e K eram justamente o par que unia as duas cópias, e o resultado é um buraco triangular (v, n, w); nas vezes seguintes o buraco já existe, os lados encontrados já são de borda e o buraco ganha um vértice (Figura 13).

4.5 Relações entre os operadores

Olhando os quatro operadores lado a lado, todos fazem a mesma coisa em essência: *dividem um leque da estrela de v entre duas "paredes", dão um vértice novo a um dos lados e decidem o que fazer com os lados curtos.* O que muda é a parede onde a rotação começa e onde ela para:

Operador	começa em	para em	lados curtos
Canônico (borda)	aresta nova	borda da malha	novo lado na borda
Expand Edge	aresta colapsada c	aresta nova	c é colada ao novo lado
Split Element	aresta colapsada c	aresta nova (após 1 elemento)	c é colada ao novo lado
Open Hole	aresta nova	primeiro elemento de interface	descolados (buraco)



(a) Antes. **(b)** Rotação a partir de c. **(c)** Depois.

Figura 12: Expand Edge em B: a aresta BC continua a trinca AB, e o elemento de interface em AB não toca os elementos de BC. (b) Ordem em que `reindexElementsCCW` visita os cantos: 1 é a aresta colapsada c (em I), 2 e 3 são os elementos abaixo da trinca; a rotação para ao encontrar he (a half-edge de BC que parte de B). (c) Os cantos visitados passam a usar B_1; A e C continuam colapsados.

Algoritmo 5 `openHole`(he, x)

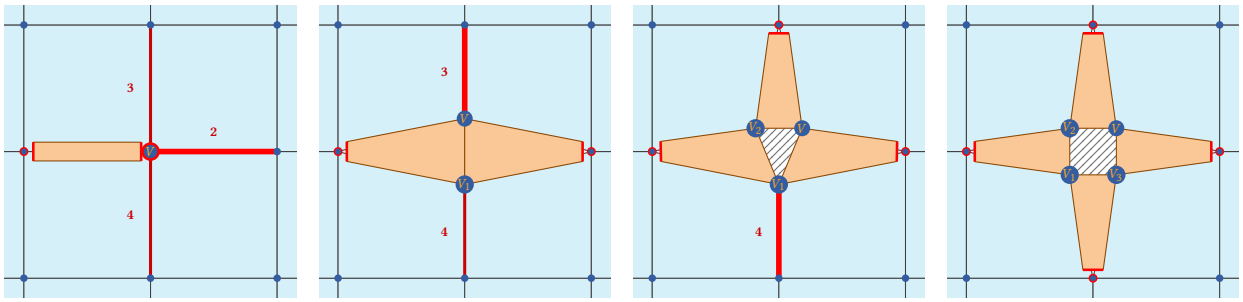
1: $v \leftarrow \mathrm{V}[he]$; $n \leftarrow$ `duplicatePoint`(v); $h \leftarrow he$
2: **repita** ▷ laço 1: anti-horário, reindexando
3: $\mathrm{V}[h] \leftarrow n$; $h \leftarrow \mathrm{O}[\mathrm{prev}(h)]$
4: **até que** $h =$ BORDER **ou** `_isInterfaceElement`$[\mathrm{elem}(h)]$
5: **assert**$(h \neq$ BORDER$)$
6: $\mathrm{V}[h] \leftarrow n$; $\mathrm{O}[\mathrm{prev}(h)] \leftarrow$ BORDER ▷ canto e lado curto de J
7: $h \leftarrow he$
8: **repita** ▷ laço 2: horário, só procura
9: $h \leftarrow \mathrm{next}(\mathrm{O}[h])$
10: **até que** $h =$ BORDER **ou** `_isInterfaceElement`$[\mathrm{elem}(h)]$
11: **assert**$(h \neq$ BORDER$)$
12: $\mathrm{O}[h] \leftarrow$ BORDER ▷ lado curto de K
13: $\mathrm{V}[x] \leftarrow n$; $\mathrm{V}[\mathrm{next}(x)] \leftarrow v$; $\mathrm{O}[x] \leftarrow$ BORDER
14: marca v e n como vértices de EI

Uma consequência interessante: o **Split Element é um caso particular do Expand Edge**. Na hipótese x_3, a rotação do Expand a partir de c visita exatamente o canto de c e o canto $\mathrm{next}(b)$ de S e para em he — as mesmas alterações do Split. Na hipótese x_1, a rotação do Expand vai pelo outro lado e reindexa todos os elementos da estrela *exceto* S; o resultado é a mesma divisão da estrela, só com os nomes v e n trocados. Verificamos isso experimentalmente: substituindo o Split pelo Expand em 791 ocorrências (400 malhas aleatórias de triângulos, quadriláteros, híbridas e com polígonos), a topologia final foi idêntica em todos os casos (Seção 6). O Split Element é, portanto, uma *otimização*: custo $O(\log n_c)$ em vez de $O(d \log n_c)$, movendo sempre o lado menor. Isso não é um defeito; mas é uma informação útil para a extensão 3D, onde talvez valha começar pela versão geral (rotação) e só depois especializar.

4.6 Contagem prévia de vértices

Para reservar memória de uma vez, `computeNumberOfNewVertices` calcula, antes de alterar a malha, quantos vértices a chamada vai criar. A contagem tem duas etapas:

1. **Seleção.** Percorre as arestas do lote e descarta as mesmas que `insertInterfaceElements` vai rejeitar, com os mesmos testes e na mesma ordem: índice que não é half-edge da malha ($h \geq Of[n_c]$), aresta de borda, aresta de elemento de interface e aresta repetida no lote (inclusive quando vem uma vez por cada uma de suas half-edges; a aresta é identificada pela menor das duas). Para cada vértice tocado pelas arestas que sobram, guarda o número n_v dessas arestas e uma half-edge que parte dele.



(a) Aresta 1: Canônico. **(b)** Aresta 2: Expand. **(c)** Aresta 3: Open Hole. **(d)** Aresta 4: Open Hole.

Figura 13: Quatro arestas chegando ao vértice interno V (detalhe do centro de uma malha 4×4; a próxima aresta a inserir está em vermelho grosso). (a) A primeira deixa V colapsado. (b) A segunda, alinhada, divide V em duas cópias coladas pelos lados curtos. (c) A terceira descola esses lados: buraco triangular (hachurado). (d) A quarta aumenta o buraco para um quadrilátero. Com k arestas, o buraco tem k vértices.

2. **Regra por vértice.** Aplica, uma vez por vértice,

$$\text{novos}(v) = \begin{cases} n_v - 1, & v \text{ fora de EI e interno (o primeiro contato só colapsa),} \\ n_v, & v \text{ fora de EI e na borda (o primeiro contato já duplica),} \\ n_v, & v \text{ já em algum EI (todo contato duplica).} \end{cases}$$

Como a regra é aplicada por vértice, e não por aresta, o teste de borda (`isBorder`, que gira em torno do vértice) roda uma vez por vértice distinto, e um vértice tocado por várias arestas do lote é contado corretamente. Por descartar as arestas de elementos de interface antes do teste de borda, a contagem também deixou de chegar ao caminho em que `isBorder` não termina (I2). O resultado é exatamente o número de vértices criados, em todos os testes (Seção 6); até a v0.3 a contagem era um limite superior (defeito I5, corrigido). Num vértice de borda com duas ou mais arestas, ela dá o valor correto (n_v), embora a inserção ainda não trate esse caso (I1). Os elementos são reservados à razão de um por aresta do lote.

5 O algoritmo de travessia

5.1 Inserção de um lote de arestas

O ponto de entrada é `insertInterfaceElements`, que recebe uma lista de half-edges (uma por aresta; qualquer uma das duas serve) e as processa na ordem dada (Algoritmo 6). Antes de alterar a malha, o método conta os vértices novos e reserva espaço para eles e para um elemento de quatro half-edges por aresta. Depois, para cada aresta, valida a entrada, abre o novo elemento a partir do gabarito da Figura 6, escolhe e aplica o operador em A e em B e, por fim, liga as arestas do meio. A Seção 2.5 mostra o efeito de cada passo nos vetores da IBHM.

Alguns detalhes de implementação merecem atenção:

- **Rejeições.** Índices que não são half-edges da malha, arestas de borda e arestas que já pertencem a um elemento de interface (por exemplo, a mesma aresta pedida duas vezes, no mesmo lote ou em chamadas diferentes) são ignorados com uma mensagem no console. Nada é devolvido a quem chamou (I7), e o espaço reservado para eles fica sem uso. O teste do índice vem primeiro, porque as consultas seguintes só valem para half-edges de elementos validados. O limite N é lido no início da chamada: os elementos criados pela própria chamada não contam, e uma half-edge deles na entrada também é rejeitada. A pré-contagem aplica os mesmos testes (Seção 4.6), por isso a contagem é exata.
- **Validação antecipada do elemento.** `commitElement(4)` grava o offset do novo elemento antes de os operadores rodarem, porque na IBHM next e prev das half-edges $h_0, \ldots, h_3$ dependem dele. Durante os operadores, as posições $O[h_0]$ e $O[h_2]$ ainda contêm o BORDER da reserva; nenhuma

Algoritmo 6 `insertInterfaceElements`(E) (classe `IBHMInterfaceElementBuilder`)

Entrada: lista E de half-edges, uma por aresta a abrir

1: $N \leftarrow$ `numberOfHalfEdges`() ▷ $Of[n_c]$ antes da chamada
2: $n_v \leftarrow$ `computeNumberOfNewVertices`(E)
3: `reserveSpaceForElements`$(4|E|, |E|)$; `reserveSpaceForNodes`(n_v); aumenta `_isInterfaceElement` e `_inInterfaceElement`
4: **para cada** $a \in E$ **faça**
5: **se** $a \geq N$ **então continue** ▷ não é half-edge da malha: rejeitada
6: $b \leftarrow \mathrm{O}[a]$
7: **se** $b =$ BORDER **ou** $b =$ COLLAPSED **então continue** ▷ aresta de borda: rejeitada
8: **se** elem(a) **ou** elem(b) é elemento de interface **então continue** ▷ rejeitada
9: $h_0 \leftarrow$ `nextAvailableHE`(); $h_1, h_2, h_3 \leftarrow h_0 + 1, h_0 + 2, h_0 + 3$
10: $e \leftarrow$ `commitElement`(4) ▷ antes dos operadores: next/prev dependem de $Of[\,]$
11: $op \leftarrow$ `retrieveOperator`(a); aplica op com (a, h_3) ▷ vértice A
12: $op \leftarrow$ `retrieveOperator`(b); aplica op com (b, h_1) ▷ vértice B
13: $\mathrm{O}[h_0] \leftrightarrow b$; $\mathrm{O}[h_2] \leftrightarrow a$ ▷ arestas do meio
14: `_isInterfaceElement`$[e] \leftarrow$ **true**

rotação chega a elas porque nenhuma half-edge antiga aponta para o novo elemento até o fim da iteração.

- **Ordem A, depois B.** O operador de B é escolhido depois de o operador de A ter sido aplicado. Como os operadores só mexem na estrela do próprio vértice, isso não altera a decisão em B.
- **a e b ainda opostas.** Enquanto os operadores trabalham, a e b continuam opostas entre si; é isso que permite que as rotações em torno de A e de B atravessem a aresta que está sendo aberta (por exemplo, o laço horário do Open Hole parte de he e passa por O[he]). Só no final a e b são religadas a h_2 e h_0.
- **Lote ou uma a uma.** Chamar o método uma vez com todas as arestas ou uma vez por aresta produz a mesma topologia (Seção 6); a diferença é só a quantidade de reservas de memória.

5.2 O exemplo completo, passo a passo

A Tabela 4 lista o operador escolhido pelo código em cada extremidade, para as 12 arestas do exemplo da apresentação, na ordem dos slides. Os 24 operadores coincidem exatamente com os anunciados nos slides 15 a 26 da apresentação [7]. A Figura 14 mostra o estado da malha depois de cada inserção, e a Figura 15 o resultado final com todos os vértices nomeados.

5.3 Propriedades observadas

Independência da ordem. O resultado final não depende da ordem de inserção, da half-edge escolhida para cada aresta, nem de as arestas virem em lote ou uma a uma. A razão é que a topologia final é determinada pelo *conjunto* de arestas: as arestas selecionadas cortam a estrela de cada vértice em setores, e cada setor termina com uma cópia própria (salvo no vértice interno com uma única aresta, que fica colapsado). Os operadores sempre dividem a estrela exatamente ao longo das arestas selecionadas, de modo que qualquer ordem leva à mesma partição. Verificamos isso para três ordens do exemplo (a dos slides, a de `main.cpp` e o lote inteiro em uma única chamada), para outras 21 ordens nos testes automatizados (Seção 6.3) e, nos 659 casos aleatórios que terminaram (Seção 6), para três ordens aleatórias e uma inserção em lote.

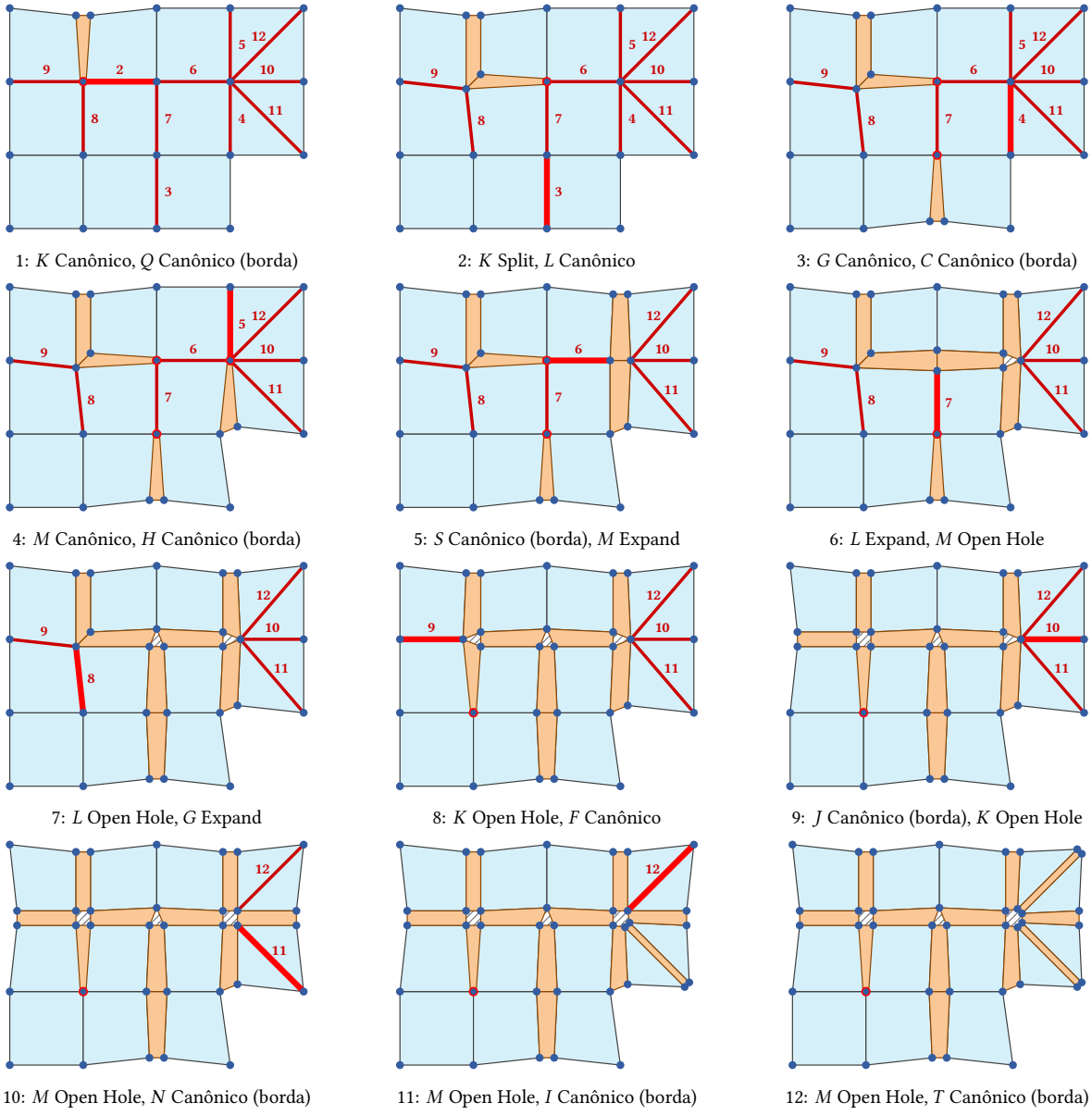


Figura 14: Estado da malha após cada uma das 12 inserções do exemplo (vista explodida). As arestas ainda não inseridas estão em vermelho e a próxima, em vermelho grosso. Observe a sequência em M: colapsado (4), dividido por Expand (5), buraco triangular (6) que cresce até um hexágono (12). Em K a segunda aresta já chega por Split Element (2), porque KL e KQ compartilham o elemento $KLRQ$.

Aresta	A–B (no código)	Operador em A	Operador em B	vértices novos
1	K–Q	Canônico	Canônico	1
2	K–L	Split Element	Canônico	1
3	G–C	Canônico	Canônico	1
4	M–H	Canônico	Canônico	1
5	S–M	Canônico	Expand Edge	2
6	L–M	Expand Edge	Open Hole	2
7	L_1–G	Open Hole	Expand Edge	2
8	K–F	Open Hole	Canônico	1
9	J–K	Canônico	Open Hole	2
10	M_1–N	Open Hole	Canônico	2
11	M_1–I	Open Hole	Canônico	2
12	M_3–T	Open Hole	Canônico	2
			total	19

Tabela 4: Operadores escolhidos pelo código no exemplo (IBHM, malha `malha2Hyb.txt`). X_i é a i-ésima cópia criada do vértice X. A coluna A–B mostra a orientação da half-edge usada (por isso a aresta 1, "QK" nos slides, aparece como K–Q).

Custo. Cada aresta custa $O\big((\deg A + \deg B)\log n_c\big)$, dominado pelas rotações (`isBorder` no Canônico, reindexação no Expand e no Open Hole); o fator $\log n_c$ vem da busca binária em $Of[\,]$ que toda consulta next/prev faz. A pré-contagem também chama `isBorder`, uma vez por vértice distinto. O custo total é proporcional ao número de arestas inseridas para malhas de valência limitada, e só depende do tamanho da malha pelo fator logarítmico.

Memória. O número de vértices novos é exatamente o previsto, inclusive com arestas rejeitadas ou repetidas no lote. É reservado um elemento por aresta do lote, e o espaço das arestas rejeitadas fica sem uso.

O exemplo no `main.cpp`. Hoje o `main.cpp` executa este construtor sobre `malha2Hyb.txt`, com as mesmas 12 arestas dos slides em outra ordem (9, 1, 2, 8, 6, 7, 3, 10, 4, 5, 11, 12, na numeração dos slides); o resultado é o mesmo da Figura 15.

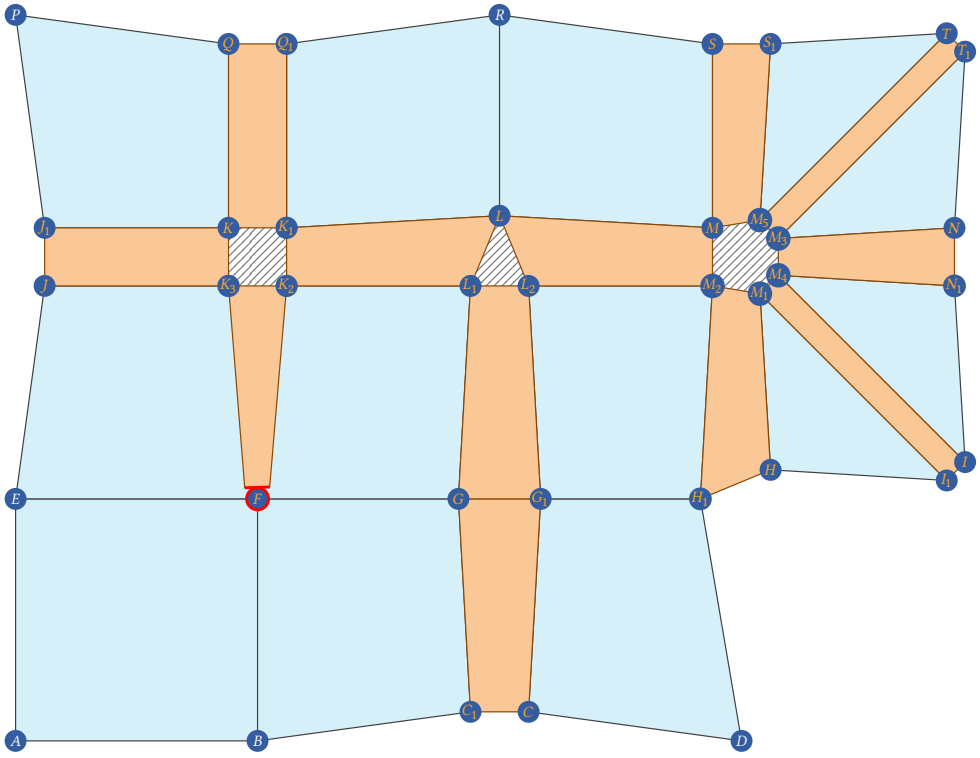


Figura 15: Resultado final do exemplo, com os nomes de todos os vértices. Os vértices K, L e M terminam com 4, 3 e 6 cópias e com buracos de 4, 3 e 6 lados; G (duas arestas) termina com duas cópias coladas; F (uma aresta, interno) continua com uma aresta colapsada (contorno vermelho); os vértices de borda C, H, I, J, N, Q, S e T foram duplicados uma vez. A posição das cópias é apenas ilustrativa: no código cada cópia tem exatamente as coordenadas do seu vértice de origem.

6 Verificação experimental

Para confirmar o entendimento descrito nas seções anteriores, escrevemos um programa de teste (`tools/driver.cpp`, Apêndice A) que usa os arquivos `IBHM.cpp` e `IBHMInterfaceElementBuilder.cpp` como estão, insere arestas (uma a uma ou em lote), registra o operador escolhido em cada extremidade e, após cada inserção, verifica:

V1. simetria das opostas: $\mathrm{O}[\mathrm{O}[h]] = h$ para toda oposta válida;

V2. consistência de vértices entre opostas: $\mathrm{V}[h] = \mathrm{V}[\mathrm{next}(\mathrm{O}[h])]$ e $\mathrm{V}[\mathrm{next}(h)] = \mathrm{V}[\mathrm{O}[h]]$;

V3. toda half-edge COLLAPSED tem comprimento zero: $\mathrm{V}[h] = \mathrm{V}[\mathrm{next}(h)]$;

V4. variedade: os cantos de cada vértice formam um único leque conexo pela relação $h \sim \mathrm{O}[\mathrm{prev}(h)]$ (nenhum vértice "estrangulado" por dois leques separados);

V5. todo vértice criado é usado por algum elemento;

V6. o número de vértices criados é igual ao previsto por `computeNumberOfNewVertices`;

V7. toda cópia tem exatamente as coordenadas do seu vértice de origem.

As verificações V1–V5 e V7 ficam em `tests/support/MeshValidation` e são as mesmas usadas pelos testes automatizados (Seção 6.3). Para comparar resultados de ordens diferentes, usamos a partição dos cantos dos elementos *originais* em classes de vértices: duas execuções produzem a mesma topologia (a menos de renumeração) se e somente se essas partições coincidem.

6.1 Malhas aleatórias

O teste com malhas aleatórias (`tools/stress.py`) gera grades de 3 a 7 por 3 a 7 células de quatro tipos — só quadriláteros, só triângulos, quadriláteros e triângulos misturados, e misturadas com hexágonos (dois quadriláteros fundidos) —, sorteia até metade das arestas internas e insere esse conjunto quatro vezes: em três ordens aleatórias, escolhendo ao acaso qual half-edge representa cada aresta, e uma vez em lote, numa única chamada. As quatro execuções precisam ser válidas e produzir a mesma topologia. Foram feitas duas rodadas de 400 malhas (Tabela 5), refeitas com o código desta versão: os números são os mesmos da v0.3, agora também com a V7.

	Rodada 1: sem restrição	Rodada 2: sem vértice de borda com 2+ arestas
Malhas (quad / tri / híbrida / polígonos)	100 / 100 / 100 / 100	100 / 100 / 100 / 100
Concluídas e válidas (V1–V7)	259	**400**
por tipo	100 / 37 / 56 / 66	100 / 100 / 100 / 100
Abortadas	141	0
das quais com vértice de borda com 2+ arestas	**141**	—
asserção em `openHole` / em `heNext`	72 / 69	—
Ordens diferentes ou lote com resultado diferente	0	0
Contagem de vértices diferente da prevista	0	0
Operadores aplicados (Canônico / Split / Expand / Hole)	3124 / 553 / 384 / 321	5130 / 791 / 708 / 535

Tabela 5: Teste com malhas aleatórias. Todas as falhas da rodada 1 são exatamente os casos em que algum vértice de borda recebe duas ou mais arestas. Nas malhas só de quadriláteros essa configuração não aparece (em uma grade, cada vértice de borda tem uma única aresta interna); nas de triângulos ela é frequente. As asserções que interrompem a execução são a do primeiro laço do `openHole` e a de `heNext`, chamada pelo segundo laço com BORDER (Seção 6.4).

6.2 Casos específicos

A Tabela 6 reúne os cenários construídos à mão: o exemplo dos slides em várias ordens, uma trinca reta e um laço fechado, o experimento que troca Split Element por Expand Edge e um conjunto de entradas inválidas. As linhas marcadas com † mudaram com as correções desde a v0.3 (I3, I4 e I5).

Em resumo: *para todas as configurações suportadas, a implementação produz topologias válidas, independentes da ordem e do modo de inserção, com contagem exata de vértices e cópias com as coordenadas do original.* Desde a v0.3, as entradas inválidas passaram a ser rejeitadas sem alterar a malha. A falha que resta é o caso de borda ainda não tratado, discutido no fim desta seção e na Seção 7.4.

6.3 Testes automatizados

Os cenários acima viraram testes automatizados do repositório (`tests/`), escritos com o Google Test e executados pelo CTest (Apêndice A). Cada teste roda em um processo próprio, com limite de tempo, de modo que uma asserção da biblioteca ou um laço infinito faz falhar só aquele teste. O núcleo é compilado de novo para os testes com as asserções ligadas, qualquer que seja o tipo de build, porque alguns defeitos só aparecem como asserção. A Tabela 7 lista as suítes; o teste com malhas aleatórias também é registrado no CTest.

Para saber se os testes detectam os defeitos que já foram corrigidos, as mesmas suítes foram compiladas contra o núcleo de versões anteriores (Tabela 8). Cada correção é detectada por mais de um teste, e o núcleo atual passa em todos.

Cenário	Resultado
Exemplo dos slides (ordem dos slides)	válido em todos os passos; 24/24 operadores iguais aos slides; 19 vértices novos (previsto: 19); buracos em K, L e M.
Exemplo, ordem do `main.cpp`, lote único e mais 21 ordens	mesma topologia da ordem dos slides.
Coordenadas das cópias (V7) †	toda cópia tem as coordenadas do original, no exemplo e nos 659 casos aleatórios válidos (até a v0.3 as cópias nasciam em $(0, 0)$).
Trinca reta J–K–L–M–N em 3 ordens	válida; mesma topologia nas 3 ordens.
Laço fechado em torno do elemento *FGLK*	válido; Split Element nos quatro cantos.
Split Element substituído por Expand Edge[a]	791 ocorrências nas 400 malhas da rodada 2: topologia idêntica nas 1600 execuções.
Mesma aresta duas vezes (em chamadas separadas ou no mesmo lote) †	a segunda é rejeitada; a pré-contagem é exata (antes reservava vértices que não eram usados).
Aresta de borda na entrada †	rejeitada; contada como 0 vértices (antes, 2).
Half-edge de um elemento de interface na entrada †	rejeitada, sem alterar a malha (antes, **laço infinito** em `isBorder`, dentro de `computeNumberOfNewVertices`).
Índice de half-edge maior que o vetor †	rejeitado, sem alterar a malha (antes, asserção em `heOpposite`; em *release*, leitura fora do vetor).
Índice no espaço reservado e ainda não validado[b] †	rejeitado; a posição reservada de O vale BORDER (antes valia 0, a aresta era aceita e a inserção terminava numa asserção do Open Hole).
Malha com furo interno (`meshIBHM.txt`): aresta 0–4 e depois 4–8	a primeira é válida (Canônico de borda nas duas pontas); a segunda falha no Open Hole, porque o vértice 4 está na borda do furo (I1).

Tabela 6: Cenários específicos. [a]Variante compilada à parte em que `retrieveOperator` nunca escolhe Split Element. [b]Por exemplo, a half-edge 48 de `malha2Hyb.txt` depois de uma chamada cuja única aresta foi rejeitada: a reserva criou as posições 48–51, mas nenhum elemento foi validado.

6.4 O caso não suportado: vértice de borda com duas arestas

Quando um vértice de borda já foi aberto pelo Canônico e recebe uma segunda aresta, ele pertence a um elemento de interface e não tem aresta colapsada; `retrieveOperator` escolhe então Open Hole. Mas um dos dois laços de `openHole` chega à borda da malha antes de encontrar um elemento de interface (Figura 16). Em modo *debug*, a asserção `currentHE != IBHM::BORDER` interrompe o programa (os comentários do código perguntam "*@todo Is it possible?*": sim, é possível). Em modo *release* o resultado é pior: o primeiro laço escreve na posição `0xFFFFFFFF` do vetor de vértices, e o segundo laço não detecta a borda, porque testa next(O[h]) em vez de O[h] — e, na IBHM, next(BORDER) é uma busca binária com um índice inválido, cujo resultado é imprevisível.

Esse caso não é raro: aparece sempre que uma trinca chega à borda por um vértice que já tem outra aresta selecionada, o que é comum em malhas de triângulos (63 das 100 malhas de triângulos da rodada 1) e nos vértices da borda de furos internos.

Suíte	Testes	O que verifica
`test_ibhm`	13	reserva exata e inclusão de pontos, `duplicatePoint` (inclusive com realocação durante a cópia), 3 coordenadas, um teste de morte (`addPoint` com número errado de coordenadas dispara a asserção), `numberOfHalfEdges` e O[ ] reservado igual a BORDER.
`test_vertex_count`	6	a pré-contagem: exemplo em lote, aresta repetida, aresta de borda, vértice já em elemento de interface, vértice de borda com duas arestas (só a contagem).
`test_base_mesh`	5	o exemplo dos slides: os 24 operadores e os vértices novos de cada inserção, com V1–V7 a cada passo; malha final; lote igual a uma por vez; duas chamadas iguais a uma; 21 ordens com a half-edge sorteada.
`test_invalid_input`	7	rejeições sem alterar a malha: índice fora do vetor, índice no espaço reservado, índice de elemento criado na mesma chamada, aresta de borda, aresta de elemento de interface, aresta já aberta, aresta repetida.
`test_known_defects`	3	desativados: I1 (uma aresta por chamada e em lote) e I2 (`isBorder`). Descrevem o comportamento correto; hoje, rodados à força, o I1 aborta no Open Hole e o I2 não termina.
`stress_supported`	1	`stress.py` com a semente 22 e 400 malhas, sem a configuração do I1 (rodada 2).
`stress_all`	1	rodada 1 (semente 21, todas as configurações); desativado até o I1 ser corrigido.

Tabela 7: Testes automatizados. Os 31 testes ativos das suítes do Google Test levam cerca de 0,2 s; o `stress_supported`, cerca de 7 s.

Núcleo	Falhas	Testes que falham
atual	0 de 31	—
antes do I3 (commit `3b991e3`)	4 de 31	O[ ] reservado; índice fora do vetor e índice de elemento da mesma chamada (asserção em `heOpposite`); índice no espaço reservado (asserção no Open Hole).
antes do I5 (`2a4eb0e`)	12 de 31	os anteriores, cinco testes de contagem e três de entrada inválida, sendo três por laço infinito.
antes do I4 (`28dd85d`)	15 de 18*	os mesmos testes de contagem e de entrada inválida e os quatro testes do exemplo que verificam a V7.

Tabela 8: Controle negativo: as suítes compiladas contra o núcleo de versões anteriores. *Sem `test_ibhm`, que usa `duplicatePoint`, inexistente nessa versão.

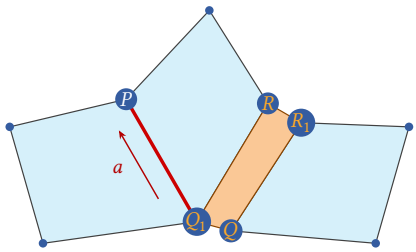


Figura 16: Configuração que falha. Q está na borda e já foi aberto pela aresta QR (cópias Q e Q_1; os dois elementos à esquerda de QR passaram a usar Q_1). Ao inserir QP, o Open Hole em Q_1 gira no sentido anti-horário a partir de *he* e alcança imediatamente a borda (à esquerda), sem encontrar elemento de interface. Nesse caso não há buraco a abrir: bastaria duplicar Q_1 e mover para a cópia nova os elementos entre QP e a borda (como no Canônico de borda), deixando o lado curto do novo elemento na borda.

7 Documentação × implementação

Esta seção confronta os comentários do código (`IBHMInterfaceElementBuilder.h` e `IBHM.h`) e o artigo da IBHM com o que o código faz, comenta os arquivos de dados e lista os defeitos em aberto. Cada item tem uma etiqueta: **[Erro]** (afirma algo falso sobre o código), **[Imprecisão]** (incompleto ou ambíguo), **[Divergência]** e **[Menor]** (redação, digitação). A apresentação de projeto [7] cumpriu seu papel e não é mais mantida; a documentação de referência passa a ser este relatório.

7.1 Comentários do código

A v0.2 apontou 14 problemas nos comentários (B1–B14). Desde então, todos foram corrigidos, exceto o B10 (Tabela 9). Os números foram mantidos para facilitar o acompanhamento.

ID	Onde	Problema na v0.2 e correção	Estado
B1	`openHole`	O parâmetro `he` dizia que o vértice tinha aresta colapsada; agora diz que o vértice já está em elemento de interface e *não* está em `_collapsed`.	corrigido
B2	`IBHM::addPoint`, `reserveSpace ForNodes`	A reserva usava `resize` e `addPoint` fazia `push_back`, gravando as coordenadas depois do bloco reservado. Corrigido no código: a reserva é um `reserve` exato (só a capacidade cresce, exatamente o pedido) e `addPoint` recebe as coordenadas num `std::vector` (vazio = origem), sem limite fixo de coordenadas.	corrigido
B3	`IBHM::_halfEdge Vertex`	Citava `_numberVerticesByElement` (da CHE); agora descreve a organização por `_offset`.	corrigido
B4	classe do construtor	"*4 operators to 3D*" virou "4 operadores em 2D, puramente topológicos, pensados para a extensão 3D".	corrigido
B5	`splitElement`	Agora descreve as duas posições do elemento compartilhado (hipóteses x_1 e x_3).	corrigido
B6	`heOpposite`, `_oppositeHalfEdge`, enum `OPPOSITE`	Documentam BORDER (-1) e COLLAPSED (-2).	corrigido
B7	`computeNumber OfNewVertices`	O comentário explicava as regras e dizia que o resultado era um limite superior. Com a correção do I5, código e comentário descrevem a contagem exata em duas etapas (Seção 4.6).	corrigido
B8	`reindexElementsCCW`	Documenta a pré-condição de não chegar à half-edge que segue uma aresta colapsada.	corrigido
B9	`insertInterface Elements`	Documenta as entradas ignoradas e a falta de relato a quem chama. O `@warning` sobre índices não validados saiu com a correção do I3; a lista de rejeições inclui os índices fora da malha.	corrigido
B10	operadores	O `@return` promete `false` em caso de falha, mas os operadores sempre devolvem `true`, e o valor é ignorado. Uma solução (verificar antes de alterar a malha e tratar o retorno com `assert`) foi testada e revertida, por acrescentar verificações ao caminho crítico; o tema será retomado.	**aberto**
B11	`numberReserved Elements`	Agora diz que devolve posições de half-edge reservadas, não elementos (o nome não mudou).	corrigido
B12	`canonical`	"*See @reindexElements*" virou "*See reindexElementsCCW()*".	corrigido
B13	`_collapsed`	Diz que o valor é a própria half-edge colapsada (h_3 ou h_1) e quando a entrada sai.	corrigido
B14	vários	Restos de cópia e digitação (cabeçalho de `IBHM.h`, `elementsList`, destrutor, parâmetro inexistente, `buildOppositesLinear`, "*bother*", "*build*").	corrigido

Tabela 9: Situação dos problemas de comentários apontados na v0.2.

7.2 Artigo da IBHM

C1. [Imprecisão no artigo] Equação (1): $\text{next}_{he}(he, c) := Of[c] + \text{circ}(he + 1, n_v(c))$. Lida literalmente, com *he* sendo o índice global, a fórmula só funciona quando $Of[c]$ é múltiplo de $n_v(c)$, o que em geral não acontece em malhas híbridas. No próprio exemplo da Figura 4 do artigo, a face 2 (quadrilátero) tem $Of = 3$ e half-edges 4 a 7; a fórmula dá $\text{next}(4) = 3 + \text{circ}(5, 4) = 4$, quando deveria dar 5. A forma correta aplica o circ ao índice local: $Of[c] + \text{circ}(he - Of[c] + 1,\ n_v(c))$. A implementação usa a forma correta (`firstHE + (he - firstHE + 1) % n`).

C2. [Imprecisão no artigo, menor] Algoritmo 4 (estrela de um vértice): como está impresso, para um vértice de borda o laço executa $he \leftarrow \text{O}[\text{prev}(he)] = 0$ e insere $\text{cell}_{he}(0)$ antes de testar a condição de parada, e a célula de $VH[v]$ só é incluída quando o laço volta a ela (vértices internos). Ao que parece, a intenção é inserir primeiro a célula de $VH[v]$ e testar $\text{O}[\text{prev}(he)] = 0$ antes de avançar. O algoritmo não é usado pela implementação.

C3. [Divergência] A implementação não tem o vetor $VH[\,]$. Por isso `isBorder` precisa girar em torno do vértice ($O(d \log n_c)$ em vez de $O(1)$), e é nessa rotação que mora o defeito I2 (Seção 7.4). Manter $VH[\,]$ atualizado pelos operadores é simples (cada operador sabe quais cantos mudou) e eliminaria as rotações de teste.

C4. [Divergência] Convenções diferentes (índices a partir de zero, -1 para borda, $n_c + 1$ offsets, estado COLLAPSED), resumidas na Tabela 1. Não são erros, mas devem ser registradas para quem comparar código e artigo.

C5. [Observação] O artigo aponta como limitação da IBHM o remalhamento dinâmico. A estratégia "contar → reservar → validar" da implementação é uma resposta a essa limitação e merece ser destacada em uma futura publicação.

7.3 Arquivos de dados

D1. [Menor] Em `malha2Hyb.txt` o vértice 18 (T) tem coordenadas $(3, 4)$; nos slides ele está em $(4, 3)$. Com $(3, 4)$, o triângulo M–T–S fica degenerado (três vértices colineares em $x = 3$). A topologia não é afetada, mas qualquer visualização ou verificação geométrica será. As figuras deste relatório usam $(4, 3)$.

D2. [Menor] `meshIBHM.txt` reproduz a Figura 4 do artigo, mas o último triângulo começa em outro vértice, o que muda a numeração de suas half-edges em relação ao artigo.

D3. [Menor] `readHybridMesh` (`main.cpp`) imprime os offsets a cada elemento lido, o que polui a saída em malhas grandes.

7.4 Defeitos e limitações encontrados no código

A Tabela 10 reúne os problemas de `IBHMInterfaceElementBuilder` e `IBHM` que continuam em aberto, confirmados com o programa de teste, com uma sugestão de correção. Como esta é a implementação que será evoluída, todos devem ser corrigidos; o de gravidade alta impede o uso em malhas gerais (de triângulos, ou com trincas que chegam à borda). Os defeitos I1 e I2 têm testes desativados que descrevem o comportamento correto (`tests/test_known_defects.cpp`, Seção 6.3): ao corrigi-los, basta reativá-los. A Tabela 11 lista os defeitos corrigidos desde a v0.3; os números foram mantidos.

ID	Grav.	Onde	Defeito e sugestão
I1	alta	`openHole`	Vértice de borda (externa ou de furo) já aberto recebendo nova aresta: asserção em debug (no primeiro laço do `openHole` ou em `heNext`, chamada pelo segundo laço com BORDER) e escrita fora do vetor em release; 141 das 400 malhas da rodada 1. *Sugestão:* permitir que os dois laços parem na borda; se o laço anti-horário parar na borda, não há canto de elemento de interface a mover; no laço horário, testar O[h] = BORDER *antes* de aplicar next. Nos vértices de borda os lados curtos já são de borda, então nada mais muda.
I2	latente	`isBorder`	Laço infinito quando a half-edge inicial vem logo após uma aresta colapsada: o desvio "pula" exatamente a half-edge de parada. Desde a correção do I5 o construtor não chega mais a esse caminho (as arestas de elementos de interface são descartadas antes do teste de borda), mas o método continua público. *Sugestão:* testar a parada também após o desvio.
I6	baixa	`retrieveAvailable Vertex`, `openHole`, `reindexElements CCW`	Os sentinelas BORDER e COLLAPSED podem chegar a next/prev (B8; segundo laço do Open Hole; COLLAPSED não é testado em x_1/x_3). Na IBHM isso vira uma busca binária com índice inválido. *Sugestão:* testes explícitos antes de cada next/prev aplicado a uma oposta.
I7	baixa	operadores, `insertInterface Elements`	Não há relato de erro: os operadores sempre devolvem `true` (B10) e o método devolve `void`; rejeições só aparecem no console. *Sugestão:* devolver a lista de arestas inseridas/rejeitadas e um estado de erro.
I9	manut.	`IBHM`	Toda consulta next/prev refaz a busca binária; nas rotações, o elemento corrente já é conhecido e o intervalo dele poderia ser reaproveitado.
I11	memória	`reserveSpace ForElements`, construtor	A política adotada é reservar memória exata. `reserveSpaceForNodes` já segue essa política, mas `reserveSpaceForElements` usa `resize`, que cresce geometricamente e pode alocar até o dobro do necessário em `_halfEdgeVertex`, `_oppositeHalfEdge` e `_offset`; o mesmo vale para `_isInterfaceElement` e `_inInterfaceElement` do construtor. *Sugestão:* um `reserve` exato antes de cada `resize`.

Tabela 10: Defeitos em aberto em `IBHMInterfaceElementBuilder` e `IBHM`, por gravidade.

ID	Defeito na v0.3	Correção e teste
I3	Entrada sem validação de índice; índice no espaço reservado aceito como aresta (O[ ] reservado valia 0).	`insertInterfaceElements` lê `numberOfHalfEdges()`, isto é, $N = Of[n_c]$, no início da chamada e rejeita todo $h \geq N$, antes de qualquer consulta; a pré-contagem aplica o mesmo teste; `reserveSpaceForElements` inicializa as posições novas de O[ ] com BORDER. Custo: uma comparação por aresta. Testes: `test_invalid_input`, `test_ibhm`.
I4	Cópias nascendo na origem.	Novo `IBHM::duplicatePoint(v)`, que acrescenta um ponto com as coordenadas de v lendo a origem por índice (correto mesmo se o vetor for realocado durante a cópia, sem vetor temporário); os quatro operadores o usam. Testes: V7 em `test_base_mesh` e no estresse; `test_ibhm`.
I5	Pré-contagem em excesso (borda, arestas rejeitadas, repetidas) e laço infinito com aresta de elemento de interface na entrada (gatilho do I2).	Contagem em duas etapas: seleção com os mesmos testes da inserção e regra aplicada uma vez por vértice (Seção 4.6). A contagem é exata. Testes: `test_vertex_count` e a V6 em todos os testes do exemplo e do estresse.
I8	Mensagem do vértice B imprimia a; `%d` com `unsigned`.	`%u` e b.
I10	Includes por inclusão transitiva; CMake exigia Qt.	Cada arquivo inclui o que usa (`<cassert>`, `<cstdio>`, `<cstring>`, `<algorithm>`); funções herdadas de C são usadas sem `std::`. O CMake não depende mais de Qt: alvos `InterfaceElementsLib`, `InterfaceElementsPrototype` e os testes em `tests/`.

Tabela 11: Defeitos corrigidos desde a v0.3.

8 Considerações finais e próximos passos

O protótipo 2D (`IBHMInterfaceElementBuilder`) cumpre o que a apresentação propôs: com quatro operadores locais, escolhidos apenas pelo estado do vértice, insere elementos de interface em qualquer ordem, sobre malhas híbridas que já contêm elementos de interface, em uma estrutura compacta baseada em índices e com memória reservada antecipadamente. A verificação experimental mostrou topologias válidas, contagem exata de vértices e independência da ordem e do modo de inserção em todas as configurações suportadas, em malhas de triângulos, quadriláteros e polígonos. Desde a v0.3, as cópias herdam as coordenadas do original, a entrada é validada, a contagem é exata e há testes automatizados que detectam cada uma dessas correções. O ponto fraco que resta está bem localizado: vértices de borda com mais de uma aresta (I1).

8.1 Um invariante que resume o algoritmo

Uma forma compacta de enunciar o que os operadores constroem, e que pode orientar tanto os testes quanto a extensão 3D, é a seguinte:

Invariante

Depois de inseridas as arestas de um conjunto S, cada vértice original v tem tantas cópias quantos são os componentes conexos da sua estrela depois de removidas as arestas de S. Cada componente usa uma única cópia.

Para um vértice interno com k arestas selecionadas, isso dá 1 cópia se $k \leq 1$ (a ponta de trinca, que fica colapsada) e k cópias se $k \geq 2$; para um vértice de borda, $k + 1$ cópias. O buraco que aparece com $k \geq 3$ é a consequência geométrica de ligar k cópias apenas por elementos de espessura nula. Como a partição em componentes não depende da ordem, qualquer algoritmo correto chega ao mesmo resultado, e é isso que explica a independência da ordem observada. Os operadores são uma forma *incremental* e local de manter esse invariante.

8.2 Rumo ao 3D

Algumas observações, como hipóteses de trabalho para a próxima etapa:

- Em 3D a entrada são faces, e o elemento de interface é um prisma (faces triangulares) ou hexaedro (faces quadrilaterais) de espessura nula. A duplicação deixa de acontecer só nos vértices: as *arestas* das faces selecionadas também precisam ser duplicadas. Uma aresta compartilhada por k faces selecionadas se comporta como um vértice 2D com k arestas selecionadas, e girar em torno de uma aresta em 3D (de célula em célula pelas half-faces opostas) é o análogo exato de girar em torno de um vértice em 2D. Os operadores 2D devem reaparecer, quase sem mudanças, como operadores *de aresta*.
- A estrela de um vértice em 3D não é mais um ciclo, e sim uma superfície fechada (ou com borda) de células. Dividir essa estrela exige uma busca em largura pelas células que não atravessa faces selecionadas, e não uma rotação. O invariante acima continua valendo e dá o critério: uma cópia por componente.
- O estado colapsado tem análogos em 3D (faces laterais degeneradas do prisma), e o buraco das junções vira "tubos" ao longo das arestas de junção.
- O sentinela COLLAPSED, a pré-contagem e a estratégia contar/reservar/validar se transportam diretamente para a 3D-IBHM.

8.3 Próximos passos sugeridos

1. **Corrigir I1**, e em seguida o I2 latente e o I6 (Tabela 10). Ao corrigir o I1, reativar os testes desativados e o `stress_all`: a rodada 1 da Tabela 5 deve passar inteira.
2. **Ampliar os testes**: a malha com furo interno (`meshIBHM.txt`), que hoje falha pelo I1, e, se valer a pena, levar o gerador de malhas aleatórias para C++, como teste parametrizado (hoje ele é o `stress.py`, chamado pelo CTest).
3. **Transferência de estado**: expor o mapa "vértice novo → vértice de origem". Numa análise adaptativa, os campos nodais (deslocamentos, poropressões etc.) dos vértices novos precisam ser copiados do vértice de origem; o construtor conhece essa informação no momento da criação (as coordenadas já são copiadas, I4).
4. **Relatar resultados** (I7): devolver quais arestas foram inseridas e quais foram rejeitadas, e por quê.
5. **Simplificar os operadores** em torno de uma primitiva comum de "divisão de leque" (Seção 4.5), mantendo o Split Element como otimização.
6. **Manter o vetor** $VH[\,]$ da IBHM, eliminando as rotações de `isBorder`, e reaproveitar o intervalo do elemento corrente nas rotações (I9).
7. **Política de memória** (I11): aplicar a reserva exata também aos vetores de elementos e aos vetores do construtor, como já é feito com as coordenadas.
8. **Retomar o B10**: decidir como os operadores devem relatar que não puderam ser aplicados sem pesar no caminho crítico.
9. **Integração com o GeMA** e, em seguida, a extensão 3D.

Referências

[1] M. Lage, L. F. Martha, J. P. Moitinho de Almeida, H. Lopes. *IBHM: index-based data structures for 2D and 3D hybrid meshes*. Engineering with Computers 33:727–744, 2017. DOI 10.1007/s00366-015-0395-0.

[2] M. Lage, T. Lewiner, H. Lopes, L. Velho. *CHE: a scalable topological data structure for triangular meshes*. Relatório técnico, PUC-Rio, 2005.

[3] M. Lage, T. Lewiner, H. Lopes, L. Velho. *CHF: a scalable topological data structure for tetrahedral meshes*. SIBGRAPI 2005, pp. 349–356.

[4] M. Mäntylä. *An Introduction to Solid Modeling*. Computer Science Press, 1988.

[5] W. Celes, G. H. Paulino, R. Espinha. *A compact adjacency-based topological data structure for finite element mesh representation*. Int. J. Numer. Methods Eng. 64:1529–1556, 2005.

[6] G. H. Paulino, W. Celes, R. Espinha, Z. Zhang. *A general topology-based framework for adaptive insertion of cohesive elements in finite element meshes*. Engineering with Computers 24:59–78, 2008.

[7] J. Coelho. *Adaptive Interface Elements Generation – 2D Case*. Apresentação (Tecgraf/PUC-Rio), versão de 25/02/2026. Arquivos `doc/ppt/Operators_Template.{pptx,pdf}`.

A Testes, ferramentas e geração das figuras

Os testes automatizados ficam em `tests/` e são compilados pelo CMake; as ferramentas do relatório ficam em `doc/report/tools/`. Todos usam o código do repositório como está e podem ser rodados a partir da raiz do repositório. Cada arquivo começa com um comentário que descreve seu uso; este apêndice explica o que cada um faz e como os resultados do relatório foram obtidos.

Arquivo	Papel
`tests/test_*.cpp`	suítes do Google Test (Tabela 7);
`tests/support/`	malhas de teste, verificações V1–V7 e acesso de leitura ao estado do construtor, usados pelos testes e pelo `driver`;
`tests/CMakeLists.txt`	registra as suítes e o teste com malhas aleatórias no CTest;
`tools/driver.cpp`	programa de teste da `IBHMInterfaceElementBuilder`: insere arestas, verifica a malha e grava cada estado em JSON;
`tools/compare.py`	compara a topologia final de execuções diferentes;
`tools/stress.py`	teste com malhas aleatórias, várias ordens e inserção em lote;
`tools/gen.py`	gera as figuras de malha do relatório (`doc/report/figs/`).

A.1 Como compilar e rodar

Os comandos abaixo partem da raiz do repositório; `$W` é um diretório de trabalho qualquer, fora do repositório. O CMake não depende de Qt. O Google Test vem do pacote instalado, se houver; senão, a versão 1.17.0 é baixada na configuração, com o hash SHA256 fixado no `CMakeLists.txt`. (No macOS, o `cmake` e o `ninja` do CLion ficam em `/Applications/CLion.app/Contents/bin/`.) O tipo de build pode ser `Debug` ou `Release`: nos dois, os testes rodam com as asserções ligadas.

```
cmake -S . -B $W/build -G Ninja -DCMAKE_BUILD_TYPE=Debug
cmake --build $W/build
cd $W/build && ctest -j8 # ou: ctest -L unit; ctest -L stress
D=$W/build/tests/driver
$D malha2Hyb.txt $W/ex.json 31 34 11 26 47 38 23 19 30 42 29 45 # exemplo dos slides
python3 doc/report/tools/stress.py $D 21 400 $W/s1 # rodada 1
python3 doc/report/tools/stress.py $D 22 400 $W/s2 --skip-border-multi # rodada 2
python3 doc/report/tools/gen.py $D doc/report/figs $W/work # figuras
cd doc/report && tectonic -X compile relatorio.tex # relatório
```

Sem CMake, o `driver` também pode ser compilado diretamente:

```
c++ -std=c++17 -Icore/TopologyDataStructures -Icore/InterfaceElementsBuilder -Itests/support \
   doc/report/tools/driver.cpp tests/support/MeshValidation.cpp \
   core/TopologyDataStructures/IBHM.cpp \
   core/InterfaceElementsBuilder/IBHMInterfaceElementBuilder.cpp -o $W/driver
```

A.2 Os testes automatizados (`tests/`)

O alvo `InterfaceElementsTests` reúne as suítes da Tabela 7, e `gtest_discover_tests` registra cada `TEST` como um teste do CTest, com o rótulo `unit` e limite de 60 s. Como cada teste roda num processo próprio, uma asserção ou um laço infinito faz falhar apenas aquele teste. O teste com malhas aleatórias entra como `stress_supported` (rótulo `stress`), que chama o `stress.py` com o `driver`; o `stress_all` está registrado, mas desativado até o I1 ser corrigido.

Asserções ligadas. Alguns defeitos só aparecem como asserção, e o teste de morte de `addPoint` espera uma. Por isso o `tests/CMakeLists.txt` retira o `NDEBUG` das opções do diretório e compila o núcleo de novo (`InterfaceElementsLibChecked`), qualquer que seja o tipo de build; a biblioteca `InterfaceElementsLib` continua compilada normalmente.

Código de apoio (`tests/support/`):

- `TestMeshes`: a malha base (`malha2Hyb.txt`, com o vértice T em $(4, 3)$) e a lista das 12 arestas do exemplo, a malha em leque do caso de borda com duas arestas e uma malha de dois triângulos;
- `MeshValidation`: a "foto" da malha (lida pela interface pública da `IBHM`, com next/prev recalculados), as verificações V1–V5, a origem de cada cópia e a V7, e a partição dos cantos usada para comparar topologias;

- `BuilderInspector`: acesso de leitura ao estado privado do construtor (o operador que seria escolhido, `_isInterfaceElement`, `_inInterfaceElement` e `_collapsed`). O construtor a declara como classe amiga. A alternativa usada até a v0.3, `#define private public`, foi abandonada: no MSVC o nível de acesso faz parte do nome decorado das funções, e a ligação falha.

Defeitos em aberto. Os testes de `test_known_defects.cpp` têm o prefixo `DISABLED_`: aparecem como desativados em toda execução e descrevem o comportamento correto, de modo que, corrigido o defeito, basta tirar o prefixo. Para rodá-los mesmo assim: `InterfaceElementsTests --gtest_also_run_disabled_tests --gtest_filter='KnownDefects*'`.

A.3 O programa de teste (`driver.cpp`)

Uso: `driver [--batch] <malha> <saida.json> <he0> <he1> ...`. A malha está no formato de `malha2Hyb.txt`: o número de vértices e de elementos, as coordenadas e, para cada elemento, o número de vértices seguido dos índices. Cada `he` é uma half-edge da aresta a abrir (qualquer uma das duas). Sem `--batch`, cada aresta é inserida numa chamada de `insertInterfaceElements`; com `--batch`, todas vão numa única chamada.

O que faz em cada chamada:

1. tira uma "foto" da malha (vértices de cada elemento, offsets e opostas), lida pela interface pública da `IBHM`;
2. se a aresta vai ser aceita, pergunta a `retrieveOperator` qual operador será usado em cada extremidade, só para registrá-lo (a decisão não muda nada na malha);
3. chama `computeNumberOfNewVertices` e `insertInterfaceElements`;
4. descobre o vértice de origem de cada vértice novo: todo vértice novo substitui, em algum canto de um elemento que já existia, o vértice que estava lá antes; esse vértice antigo (ou a origem dele) é a origem do novo;
5. aplica as verificações V1–V5 e V7 da Seção 6 e compara o número de vértices criados com o previsto (V6);
6. imprime uma linha e grava o estado no JSON.

As verificações são feitas sobre a "foto", com consultas next/prev recalculadas, para não depender do código que está sendo testado; a foto, as verificações e a origem das cópias vêm de `tests/support/MeshValidation`, as mesmas dos testes automatizados. O estado privado do construtor (`_collapsed`, `_isInterfaceElement` e o operador escolhido) é lido por `BuilderInspector`. Na v0.4 o `driver` passou a usar esse código compartilhado; sua saída (console e JSON) continuou idêntica, byte a byte, no exemplo, nos casos de entrada inválida e em 834 execuções do estresse (120 malhas de cada rodada, inclusive as que abortam).

Saída no console: uma linha por chamada, como

```
step 2 edge he=34 (10-11) A:SPLIT_ELEMENT B:CANONICAL new=1 predicted=1 ok
```

com a half-edge, os dois vértices da aresta, o operador de cada extremidade, os vértices criados e previstos e o resultado (`ok` ou `INVALID`, seguido das verificações que falharam). O código de saída é 0 se tudo passou e 2 se alguma verificação falhou; uma asserção da biblioteca interrompe o programa.

JSON: um objeto com `points` (coordenadas originais) e `steps`, uma lista em que o passo 0 é a malha de entrada e o passo k é o estado depois da k-ésima chamada. Cada passo tem:

Campo	Conteúdo
`edge, A, B`	a half-edge inserida e os vértices da sua aresta
`opA, opB`	o operador de cada extremidade (vazio no modo em lote ou para arestas rejeitadas)
`predicted, created`	vértices previstos por `computeNumberOfNewVertices` e vértices criados
`errors, msgs`	número de problemas e mensagens das verificações
`elements, first`	os vértices de cada elemento e a primeira half-edge de cada elemento (os offsets)
`opp`	a oposta de cada half-edge; -1 é BORDER e -2 é COLLAPSED
`isIE`	1 para os elementos de interface
`root`	para cada vértice, o vértice original de que ele é cópia
`collapsed`	o mapa `_collapsed` do construtor, como pares (vértice, half-edge)

O arquivo é gravado e esvaziado (*flush*) a cada passo; se um passo posterior aborta, os anteriores continuam legíveis (o `gen.py` usa isso para desenhar o caso de borda que falha).

A.4 Comparação de topologias (`compare.py`)

Duas execuções que inserem as mesmas arestas em ordens diferentes criam os vértices novos em ordens diferentes, então os índices não servem para comparar. O `compare.py` olha só para os elementos que já existiam na malha de entrada (eles mantêm seus índices) e agrupa os cantos desses elementos pelo vértice que usam. O conjunto de grupos é uma partição dos cantos; duas execuções têm a mesma topologia, a menos de renumeração, se e somente se as partições são iguais. Os elementos de interface ficam determinados pelos elementos contínuos em volta e não precisam entrar na comparação. Pela linha de comando, `python3 compare.py a.json b.json ...` diz, para cada arquivo, se a partição é igual à do primeiro.

A.5 Teste com malhas aleatórias (`stress.py`)

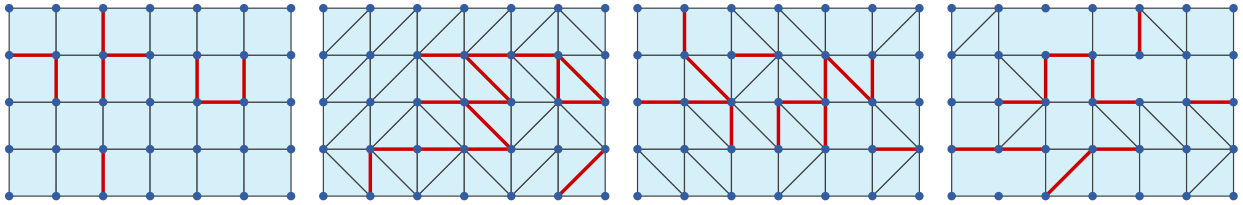
Uso: `stress.py <driver> <semente> <casos> <diretório> [--skip-border-multi]`. Cada caso:

1. gera uma malha numa grade de 3 a 7 por 3 a 7 células, alternando quatro tipos (Figura 17): só quadriláteros, só triângulos (cada célula dividida por uma das diagonais, ao acaso), híbrida (60% das células viram triângulos) e com polígonos (como a híbrida, mas 25% das vezes duas células vizinhas viram um hexágono);
2. sorteia até metade das arestas internas;
3. insere o mesmo conjunto quatro vezes com o `driver`: três vezes uma aresta por chamada, em ordens aleatórias e com a half-edge de cada aresta escolhida ao acaso, e uma vez em lote;
4. exige que as quatro execuções sejam válidas, que a contagem de vértices seja sempre a prevista e que as quatro partições sejam iguais.

Um caso é marcado como "vértice de borda com 2+ arestas" quando a seleção tem um vértice de borda tocado por duas ou mais arestas (a configuração do defeito I1). Com `--skip-border-multi`, essas seleções são sorteadas de novo. No fim, o script imprime os contadores (casos válidos, inválidos e abortados, quantos abortados tinham a configuração do I1, casos dependentes da ordem, casos em que o lote difere, contagens erradas, operadores aplicados), a asserção que causou cada aborto e o número de casos válidos por tipo de malha. O código de saída é 0 se todos os casos passaram e 1 se houve aborto, caso inválido, dependência da ordem ou do lote, ou contagem errada; é ele que o CTest usa. A semente fixa o gerador aleatório: a mesma semente sempre gera as mesmas malhas, seleções e ordens (o relatório usa as sementes 21 e 22, com 400 casos cada).

A.6 Geração das figuras (`gen.py`)

Uso: `gen.py <driver> <diretório-das-figuras> <diretório-de-trabalho>`. Para cada figura, o script monta uma malha pequena (ou lê uma do repositório), roda o `driver`, carrega o estado desejado e o con-



(a) quad **(b)** tri **(c)** hybrid **(d)** poly

Figura 17: Um exemplo de cada tipo de malha gerada pelo `stress.py` (grade de 6×4 células), com uma seleção de arestas em vermelho. As malhas e as seleções do teste usam o mesmo gerador, com tamanhos e sementes variados.

verte num `tikzpicture`, gravado em `figs/<nome>.tex`. Assim, toda figura de malha mostra o que o código realmente produz; as figuras não devem ser editadas à mão, e sim regeneradas. Três detalhes do desenho:

- **Vista explodida.** As cópias de um vértice coincidem por definição (o código lhes dá as coordenadas do original). Para a topologia ficar visível, cada cópia de um vértice dividido é desenhada um pouco afastada da posição original, na direção média dos centróides dos elementos contínuos que a usam; como cada cópia é usada pelos elementos de um setor, as cópias se afastam, cada uma para o seu setor.
- **Arestas colapsadas.** Os dois cantos de um elemento de interface sobre uma aresta colapsada são afastados na direção do elemento vizinho, e a aresta é desenhada como um pequeno segmento vermelho com espessura apenas visual.
- **Buracos.** As half-edges de borda formam laços; um laço percorrido no sentido horário (área com sinal negativa) é um buraco e é hachurado.

Os nomes dos vértices seguem a regra das figuras: o vértice original mantém o nome e suas cópias recebem índices (K, K_1, K_2, ...) na ordem de criação. As figuras desenhadas à mão (gabarito, árvore de decisão, estados, hipóteses x_1/x_3) ficam no próprio texto, em TikZ, e usam os estilos de `styles.tex`.